CMSC424: Database Design

Module: Query Processing and Optimization

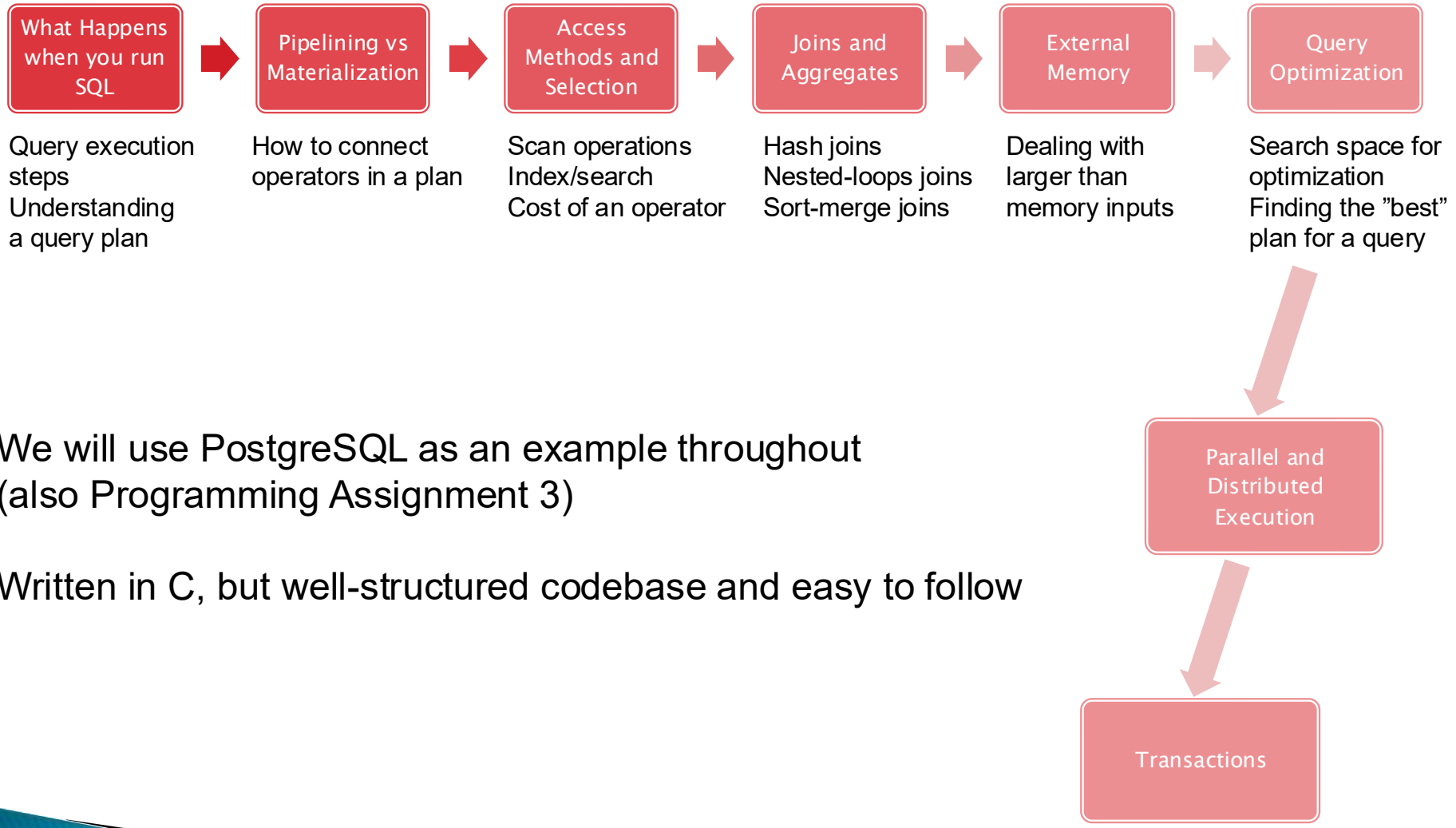
Instructor: Amol Deshpande
amol@cs.umd.edu

Database Implementation

- Shifting into discussing the internals of a DBMS
  - How data stored? How queries/transactions executed?

- Some of the Topics:
  - Storage: How is data stored? Important features of the storage devices (RAM, Disks, SSDs, etc)
  - Tuple Organization: How are tuples laid out
  - Indexes: How to quickly find specific tuples of interest (e.g., all 'friends' of 'user0')
  - Query processing: How to execute different relational operations? How to combine them to execute an SQL query? How to do this in a parallel setting?
  - Query optimization: How to choose the best way to execute a query?
  - Transactions: How to support the ACID properties? In a distributed setting?

Plan



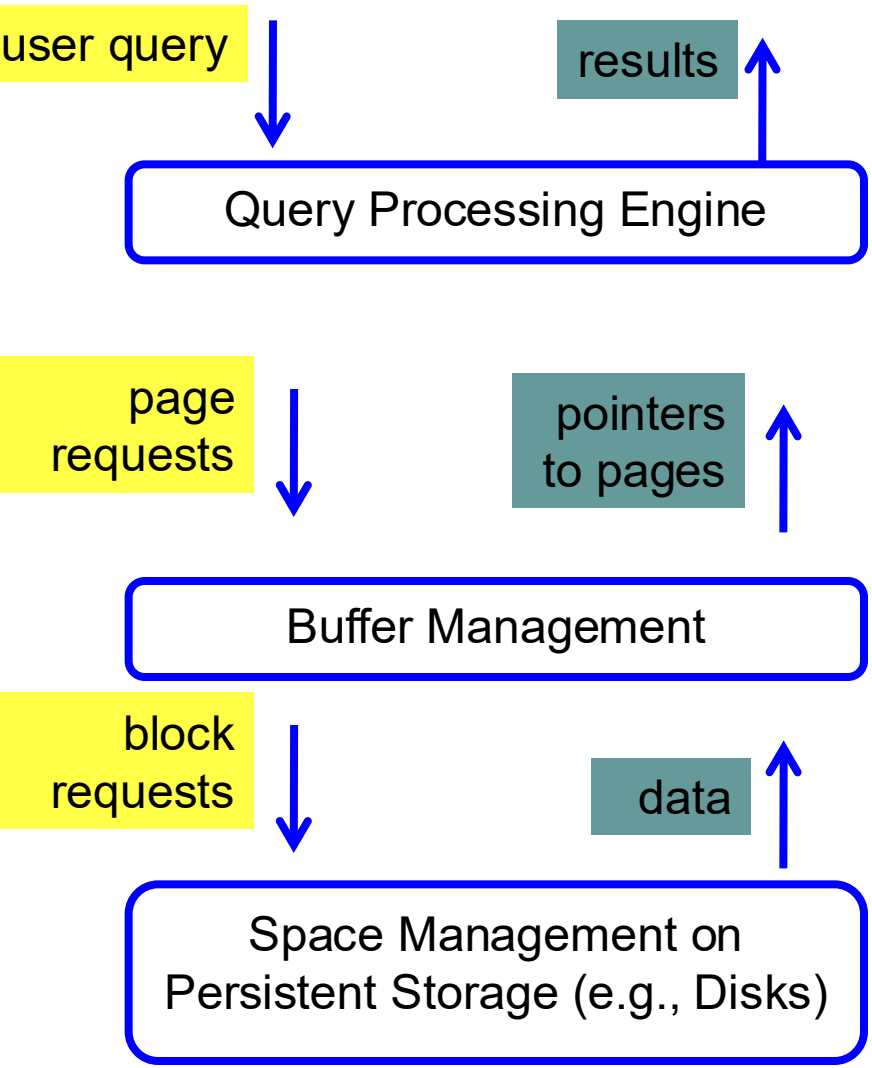
We will use PostgreSQL as an example throughout (also Programming Assignment 3)

Written in C, but well-structured codebase and easy to follow

Learning Goals

- Understand, draw, and reason about a query plan
- Reason about pipelining vs materialization for chaining of operators, and what makes sense under what conditions
- Understand how typical operators like joins, aggregates, etc., are implemented, both in centralized systems as well as parallel/distributed systems
- Design new algorithms or implementations of standard or new relational operators
- Reason about “costs” of different operators under different computing environments
- Understand query optimization issues, including equivalence between different plans

Query Processing/Storage

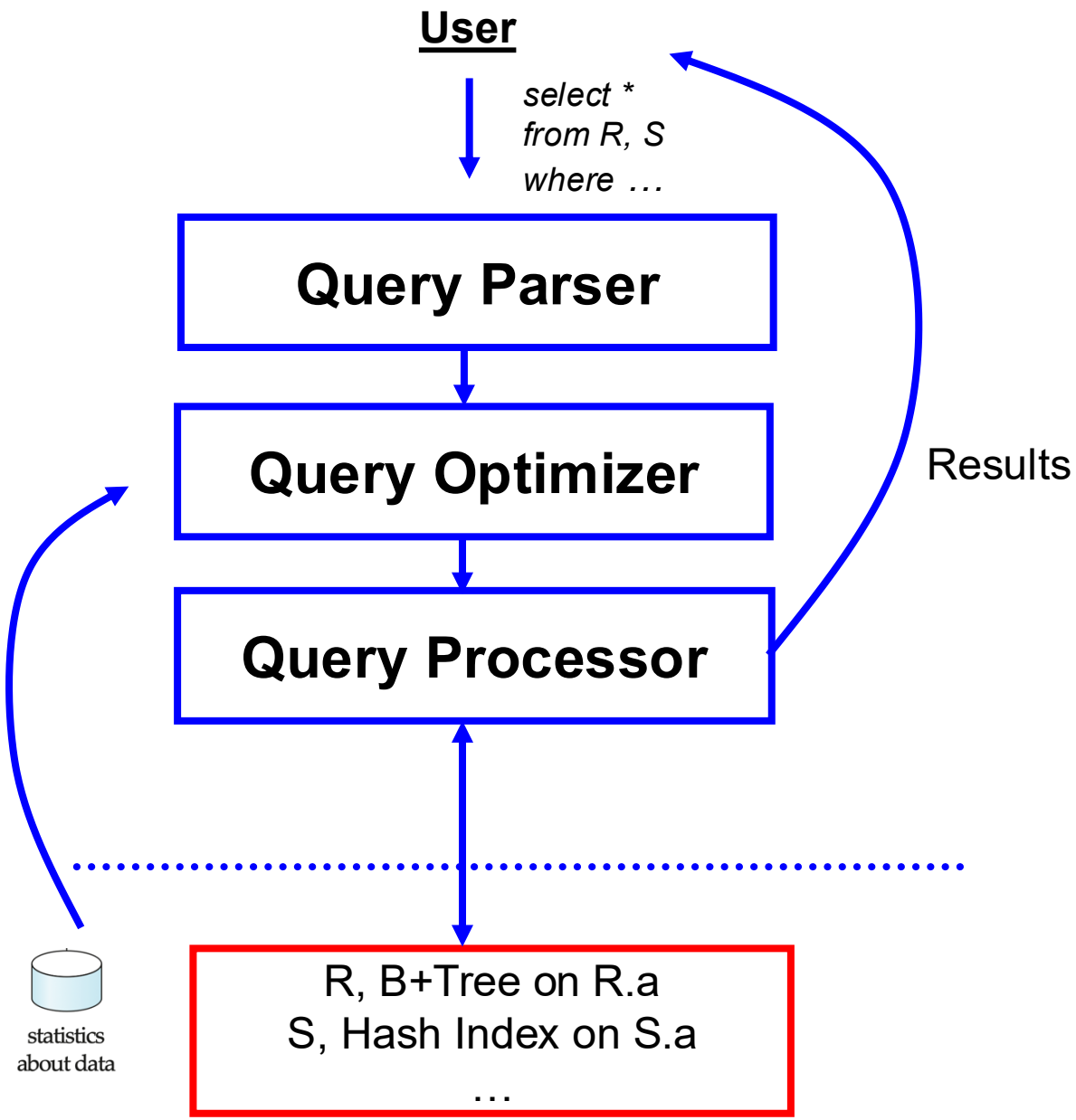


- Given an input user query, decide how to "execute" it
- Specify sequence of pages to be brought in memory
- Operate upon the tuples to produce results

- Bringing pages from disk to memory
- Managing the limited memory

- Storage hierarchy
- How are relations mapped to files?
- How are tuples mapped to disk blocks?

Getting Deeper into Query Processing



Resolve the references,
Syntax errors etc.
Converts the query to an internal format
relational algebra like

Find the *best* way to evaluate the query
Which index to use ?
What join method to use ?
…

Read the data from the files
Do the query processing
joins, selections, aggregates
…

CMSC424: Database Design

Module: Query Processing and Optimization

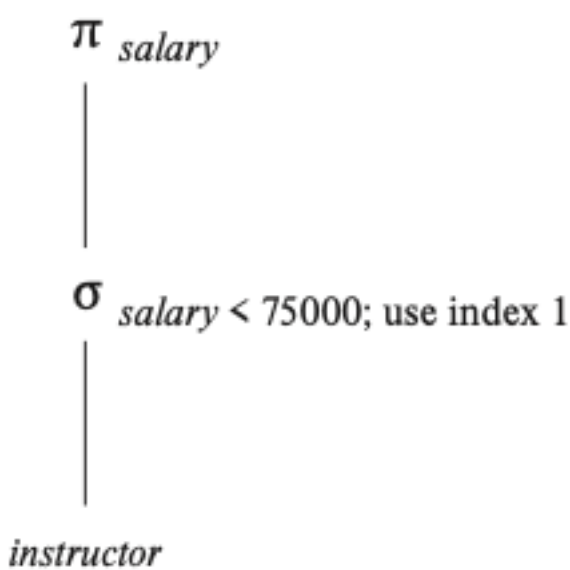
Query Plans

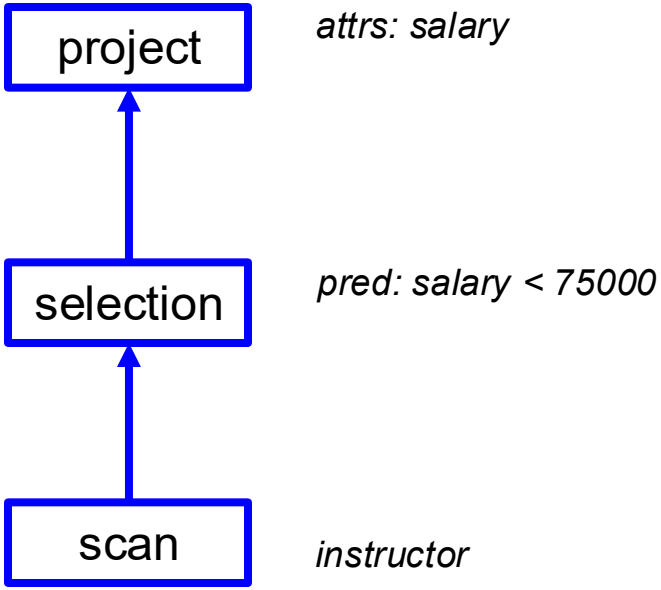
Instructor: Amol Deshpande
amol@umd.edu

Query Plan

- A tree of operators instantiated for a query
  - … with varying level of detail depending on the stage
  - … logical vs physical operators depending on the stage

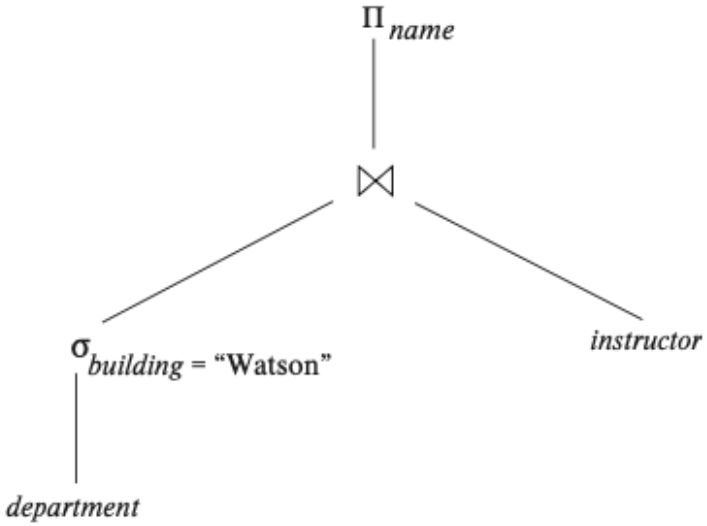
select salary
from instructor
where salary < 75000



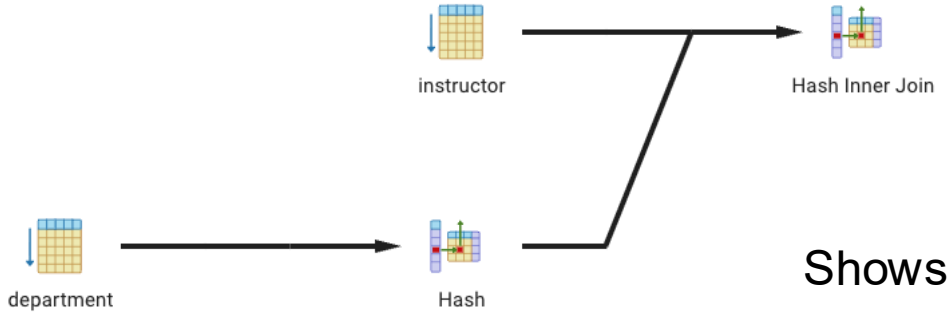


Query Plan

select name
from department natural join instructor
where building = “Watson”



A PostgreSQL execution plan



Shows “physical” operators
e.g., no explicit σ (select) or π (project)

Physical vs Logical Operators

- Logical operators: Select, Project, Natural Join, Outerjoin, Aggregate, etc.
  - Match the query language or the abstractions

- Implementation driven by other factors, especially code modularity, reuse, etc.
  - e.g., ”filter” or ”project” don’t really need to be separate physical operators – too simple for that
  - Each join variants (inner join, outer join, semi-join) doesn’t need to be a separate operator – same code can handle them
  - ”Sorting” is used by different logical operators → Should be a separate operator reused by all

- → logical operators don’t map one-to-one with physical operators

Query Plan Equivalence

- Different ways to evaluate the same query + Different ways to evaluate the same operator ➔ Need for "query optimization"

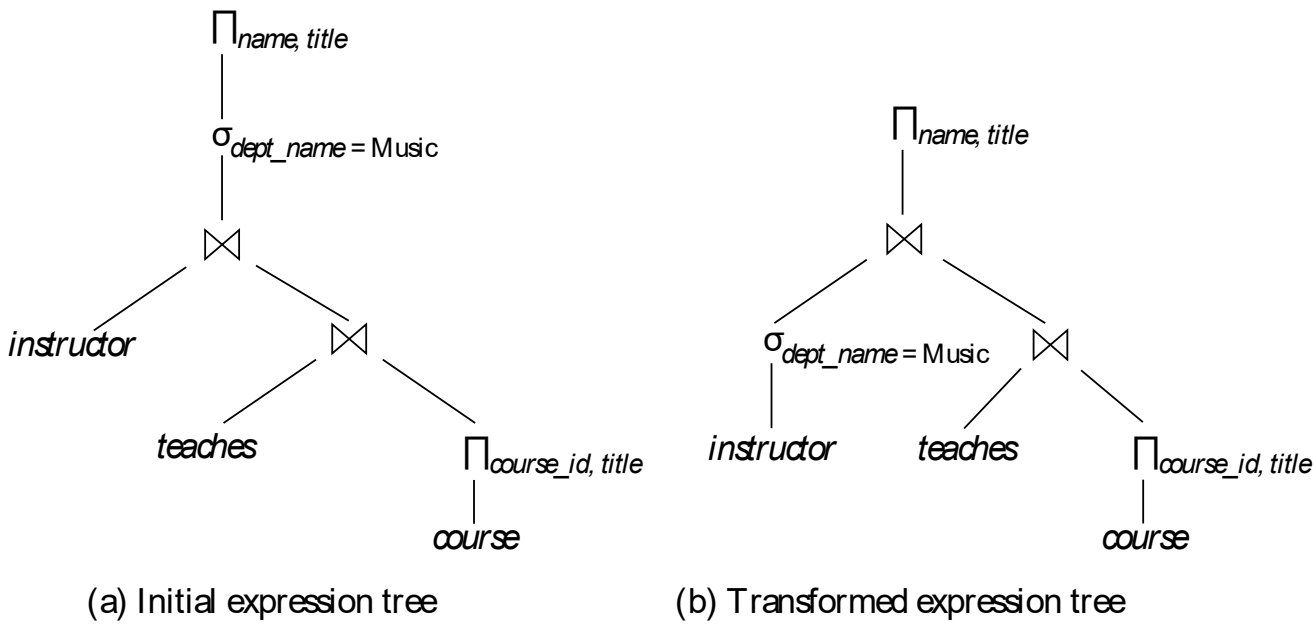
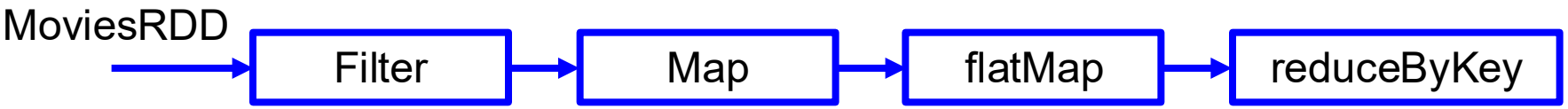


Figure 16.1 Equivalent expressions.

Spark or MongoDB Plans?

```
t3 = moviesRDD.filter(lambda t: re.search(r'\((\d{4})\)', t[1]) is not None) \
     .map(lambda t: (re.search(r'\((\d{4})\)', t[1]).group(1), t[2])) \
     .flatMap(lambda t: [((t[0], x), 1) for x in t[1].split('|')]) \
     .reduceByKey(lambda a, b: a + b)
```



- Natural mapping from the query language to the operator tree
- Less scope for “optimizations” across the operators
  - i.e., none to very few options to rearrange the operators
  - primarily because of the python functions (treated as black-boxes)
- Each operator itself needs to be optimized

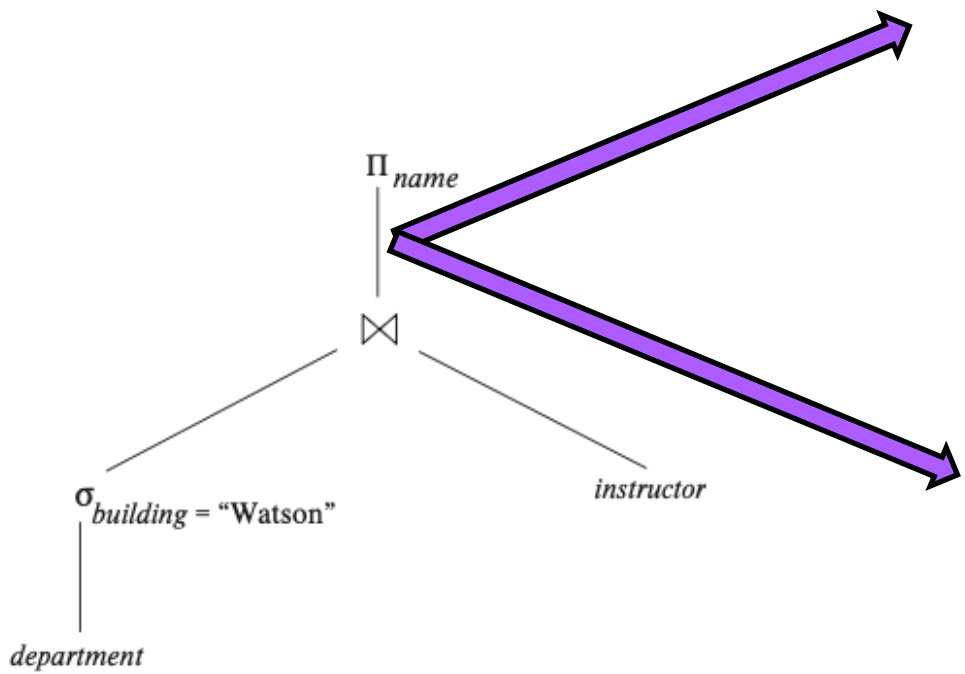
CMSC424: Database Design

Module: Query Processing and Optimization

Pipelining vs Materialization

Instructor: Amol Deshpande
amol@umd.edu

How to Pass Tuples



Materialize?

- Write out the results to a file, and next operator reads it from the file

VS

Pipeline?

- Both operators running at the same time
- Outputs from one directly passed to the other

Materialization

- Materialized evaluation is always applicable
- Cost of writing results to disk and reading them back can be quite high
- Naturally fault-tolerant
- Low memory requirements
- Often the first implementation

Pipelining

- Evaluate several operations simultaneously, passing the results of one operation on to the next.
- Cheaper: no need to store a temporary relation to disk.
- Requires higher amount of memory
  - All operations are executing at the same time (say as processes)
- Somewhat limited applicability

- A “blocking” operation: An operation that has to consume entire input before it starts producing output tuples

Operator Implementations

- Materialization
  - Relatively easy – each operator implemented to read from files and write to files
  - Pseudocode for a simple “join” operator: read from two files R and S, and write output to Out

```
open R
open Out for writing

for each tuple r in R:
    open S
    for each tuple s in S:
        if join_match(r, s):
            write joined_tuple(r, s) to Out
    close S

close R
close Out
```

Operator Implementations

- Demand-driven Pipelining using “iterator” model
  - Used in systems like PostgreSQL
- Each operator implements three functions: Open(), Next(), Close()

A ”select” operator

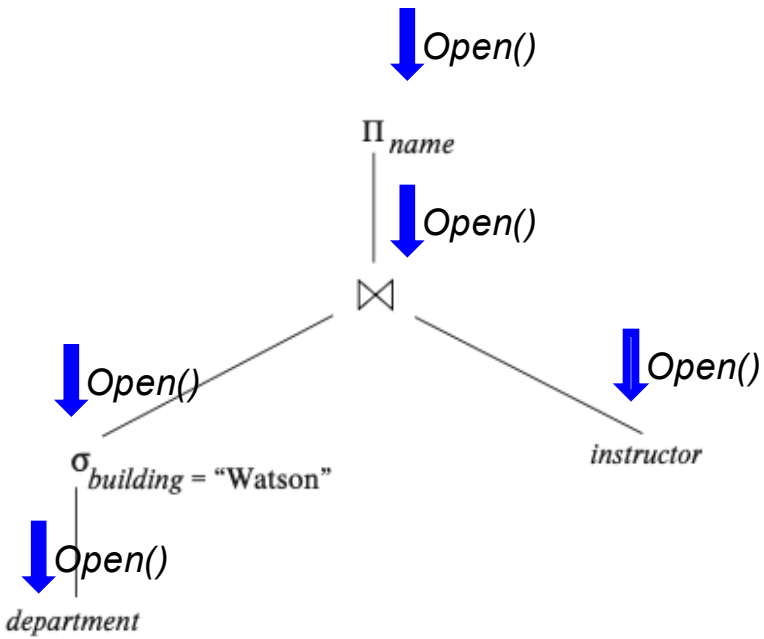
```
Select.open():
    Input.open()

Select.next():
    while True:
        t = Input.next()

        if t == EOF:
            return EOF

        if predicate(t):
            return t

Select.close():
    Input.close()
```



Operator Implementations

- Demand-driven Pipelining using “iterator” model
  - Used in systems like PostgreSQL
- Each operator implements three functions: Open(), Next(), Close()

A ”select” operator

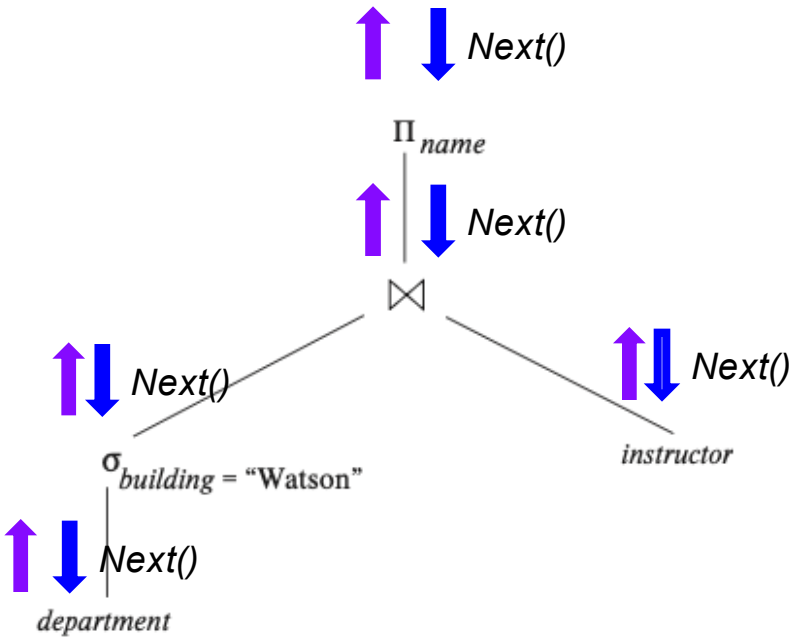
```
Select.open():
    Input.open()

Select.next():
    while True:
        t = Input.next()

        if t == EOF:
            return EOF

        if predicate(t):
            return t

Select.close():
    Input.close()
```



Operator Implementations

- Producer-drive Pipelining
  - Operators read and write to queues
  - Need to be careful in handling overfull queues
  - Easy to handle parallel setups → the queues can span machines

```
Join.run():
    r = RQ.get()
    while r != EOF:
        s = SQ.get()

        while s != EOF:
            if join_match(r, s):
                OutQ.put(join_tuple(r, s))
            s = SQ.get()

        SQ.reset()        # start S again for next r
        r = RQ.get()

    OutQ.put(EOF)
```

“Cost”

- Complicated to compute, but very important to decide early on
  - Need to know what you are “optimizing” for
- Many competing factors in today’s computing environment
  - CPU Instructions
  - Disk I/Os
  - Network Usage – either peak or average (for distributed settings)
  - Memory Usage
  - Cache Misses
  - … and so on
- Want to pick the one (or combination) that’s actually a bottleneck
  - No sense in optimizing for “memory usage” if you have a TB of memory and a single disk
  - Can do combinations by doing a weighted sum: e.g., 10 * Memory + 50 * Disk I/Os

“Cost”

- We will focus on counting the amount of data processed (in terms of disk blocks), and the CPU costs if appropriate
  - Somewhat of a simplification, but works okay
  - In particular, it doesn’t account for random vs sequential access, or the tuple layouts (mostly)

- Textbook uses a cost function that considers random vs sequential disk I/Os

CMSC424: Database Design

Module: Query Processing and Optimization

Operator Implementations

Instructor: Amol Deshpande
amol@umd.edu

Selection Operation

select * from person where SSN = “123”

```
def open(self):
    self.tableScan = TableScan("person")
    self.tableScan.open()

def next(self):
    while True:
        t = self.tableScan.next()

        if t is EOF:
            return EOF

        if t.SSN == "123":
            return t

def close(self):
    self.tableScan.close()
```

These would be initialized based on the query

Selection Operation

```
def open(self, tablename, pos, op, const):
    self.tableScan = TableScan(tablename)
    self.tableScan.open()

def next(self):
    while True:
        t = self.tableScan.next()

        if t is EOF:
            return EOF

        if evaluate(op, t[pos], const):
            return t

def close(self):
    self.tableScan.close()
```

A helper function that can handle different operators like =, <=, >=,!=, etc., and also different datatypes (e.g., integers, strings)

Cost?

 # blocks read = b_tablename

CPU cost = O(|table_name|)

Selection: B+-Tree

select * from person where SSN = “123”

```
def open(self):
        self.indexScan = BTreeLookup(index="person.SSN", key="123")
        self.indexScan.open()

def next(self):
        rid = self.indexScan.next()

        if rid is EOF:
                return EOF

        t = FetchRecord("person", rid)
        return t

def close(self):
        self.indexScan.close()
```

Cost?

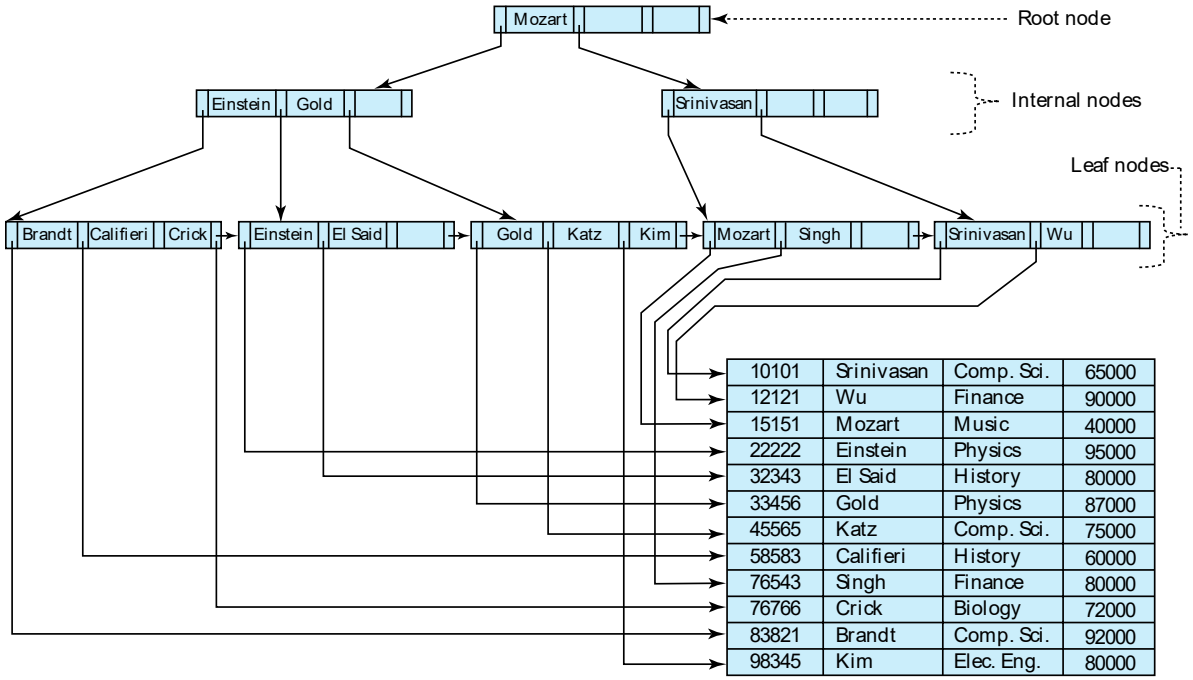
 # blocks read = h_index (height of the index) + 1

CPU cost = O(h_index + 1)

Selection Operation

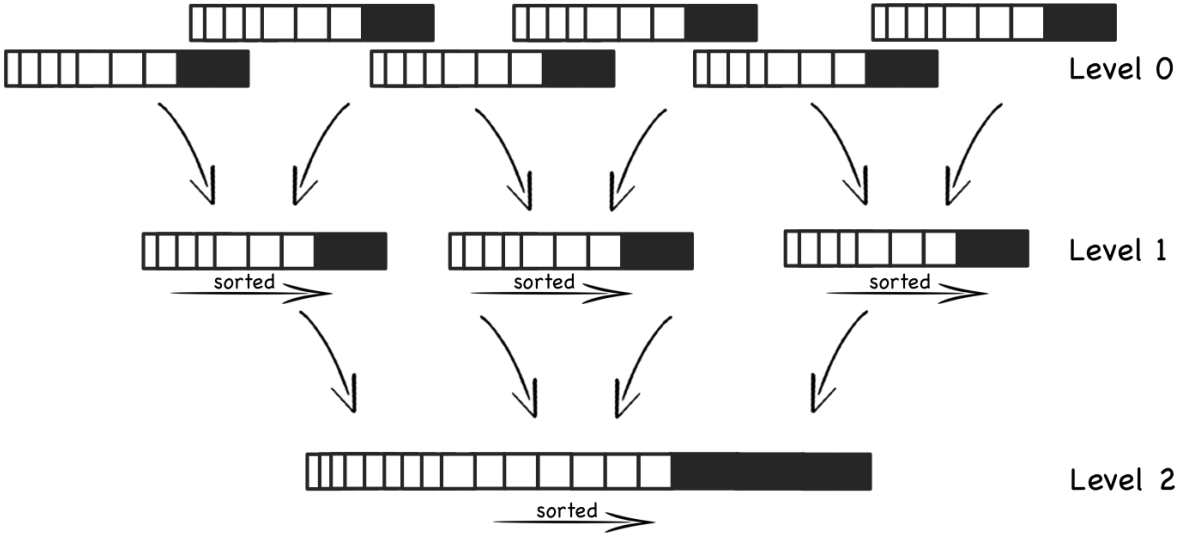
- For secondary indexes, each “FetchRecord” requires a block access
  - Can get expensive
  - You should be able to reason through a data structure and estimate the costs of such an operation

select * from instructor where name = “Singh”



Selection Operation

- Each index structure needs separate code as well as estimates
- e.g., LSMT – need to estimate the possibility to having to read the runs on disks



Compaction continues creating fewer, larger and larger files

Selection Operation

- Selections involving ranges
  - *select * from accounts where balance > 100000*
  - *select * from matches where matchdate between '10/20/06' and '10/30/06'*
  - Option 1: Sequential scan
  - Option 2: Using an appropriate index
    - Can't use hash indexes for this purpose

- Multi-attribute selections (a = 1 and b = 2)
  - Different ways to execute, including "index-ANDing" (does not require a separate operator)

Joins

- *select * from R, S where R.a = S.a*
  - Called an "*equi-join*"
- *select * from R, S where |R.a – S.a | < 0.5*
  - *Not an "equi-join"*

- *Goal: For each tuple r in R, find all "matching" tuples in S (or vice versa)*

- Simplest Algorithm ("nested loops" join)

  for each tuple r in R

  for each tuple s in S

  check if r.a = s.a (or whether |r.a – s.a| < 0.5)

- Complexity too high -- $O(N_r N_s)$
  - e.g., imagine if |R| and |S| both in millions of tuples

Index Nested-loops Join

- *select * from R, S where R.a = S.a*
  - Called an "*equi-join*"

- Let's say there is an "index" on S.a

  for each tuple r in R

  use the index to find S tuples with S.a = r.a

- Works pretty well if the index on S is a "primary" index
  - Otherwise the disk I/Os may become too much

Index Nested-loops Join

- Restricted applicability
  - An appropriate index must exist
  - What about $|R.a - S.a| < 5$?

- Great for queries with joins and selections

  *select **

  from accounts, customers

  where accounts.customer-SSN = customers.customer-SSN and

  accounts.acct-number = "A-101"

- Only need to access one SSN from the other relation

Hash Join

- <u>Assume S is the smaller relation and fits in memory</u>

  read S in memory and build a hash index (hash table) on it

  for each tuple r in R

  use the hash index on S to find tuples such that S.a = r.a

- Cost: O(N_r + N_s + *O*) -- *where O is the number of output tuples*

- Why good ?
  - CPU cost is much better

Hash Join

```
def open(self):
    self.H = build_hash_table(on=”S", key="a")
    self.rScan = TableScan(”R")
    self.rScan.open()
    self.matches = []
    self.r = None

def next(self):
    while True:
        if self.matches is not empty:
            s = self.matches.pop()
        return join(self.r, s)

        self.r = self.rScan.next()
        if self.r is EOF:
            return EOF

        self.matches = self.H.lookup(self.r.a)

def close(self):
    self.rScan.close()
```

Hash table on S built here
Usually it is built in the first call to “next”
principle: don’t touch data in open()

For each tuple in r, get all matches here, but return them one-by-one

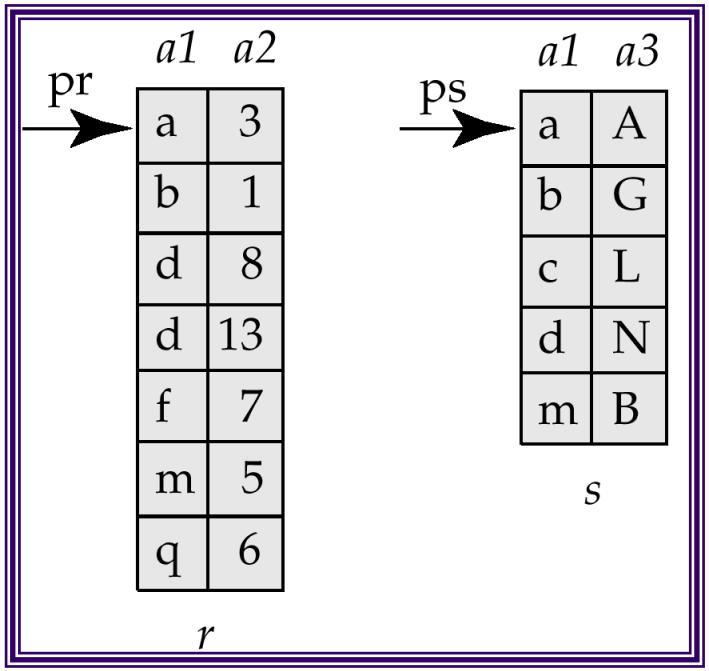
Merge-Join (Sort-merge join)

- Pre-condition:
  - The relations must be sorted by the join attribute
  - If not sorted, can sort first, and then use this algorithms
- Called “sort-merge join” sometimes

*select **
from r, s
where r.a1 = s.a1

Step:

1. *Compare the tuples at pr and ps*
2. *Move pointers down the list*
   - *Depending on the join condition*
3. *Repeat*



Merge-Join (Sort-merge join)

- Cost:
  - If the relations sorted, then just
    - $b_r + b_s$ block transfers, some seeks depending on memory size
  - What if not sorted ?
    - Then sort the relations first
    - In many cases, still very good performance
    - Typically comparable to hash join
- Observation:
  - The final join result will also be sorted on *a1*
  - This might make further operations easier to do
    - E.g. duplicate elimination

Joins: Summary

- Nested-loops join
  - Can always be applied irrespective of the join condition
  - e.g., |R.a – S.a| < 1
- Index Nested-loops join
  - Only applies if an appropriate index exists
- Hash joins – only for equi-joins
  - Join algorithm of choice when the relations are large
- Sort-merge join
  - Very commonly used – especially since relations are typically sorted
  - Sorted results commonly desired at the output
    - To answer group by queries, for duplicate elimination, because of ASC/DSC

Group By and Aggregation

select a, count(b)
from R
group by a;

- Hash-based algorithm
- Steps:
  - Create a hash table on *a,* and keep the *count(b) so far*
  - Read *R* tuples one by one
  - For a new *R tuple, "r"*
    - Check if *r.a* exists in the hash table
    - If yes, increment the count
    - If not, insert a new value

Group By and Aggregation

```
def open(self):
    self.rScan = TableScan("R")
    self.rScan.open()
    self.H = HashTable()
    self.groups = None

def next(self):
    // FIRST CALL
    if self.groups is None:
        while True:
            r = self.rScan.next()
            if r is EOF:
                break
            self.H[r.a] = self.H.get(r.a, 0) + 1

        self.groups = self.H.entries()

    // FIRST AND LATER CALLS
    if self.groups is empty:
        return EOF

    return self.groups.pop()

def close(self):
    self.rScan.close()
```

End result: H[r.a] = count

Group By and Aggregation

```
def open(self):
    self.rScan = TableScan("R")
    self.rScan.open()
    self.H = HashTable()
    self.groups = None

def next(self):
    // FIRST CALL
    if self.groups is None:
        while True:
            r = self.rScan.next()
            if r is EOF:
                break
            self.H[r.a] = self.H.get(r.a, 0) + 1

        self.groups = self.H.entries()

    // FIRST AND LATER CALLS
    if self.groups is empty:
        return EOF

    return self.groups.pop()

def close(self):
    self.rScan.close()
```

SUM(B)
self.H[r.a] = self.H.get(r.a, 0) + r.B

MIN(B)
self.H[r.a] = min(self.H.get(r.a, INFINITY), r.B)

MAX(B)
self.H[r.a] = max(self.H.get(r.a, -INFINITY), r.B)

Group By and Aggregation

```
def open(self):
    self.rScan = TableScan("R")
    self.rScan.open()
    self.H = HashTable()
    self.groups = None

def next(self):
    // FIRST CALL
    if self.groups is None:
        while True:
            r = self.rScan.next()
            if r is EOF:
                break
            self.H[r.a] = self.H.get(r.a, 0) + 1

        self.groups = self.H.entries()

    // FIRST AND LATER CALLS
    if self.groups is empty:
        return EOF

    return self.groups.pop()

def close(self):
    self.rScan.close()
```

How about AVERAGE(B)?
Need to maintain sum and count for each group

self.H[r.a] = self.H.get(r.a, (0, 0)) + (1, r.B)

(assume element-wise addition of tuples)

e = self.groups.pop(0)
return (e[0], e[1][1]/e[1][0])

How about MEDIAN(B)?
Need to collect all the values
self.H[r.a] = self.H.get(r.a, []).append(B)

e = self.groups.pop()
return (e[0], median(e[1]))

Group By and Aggregation

select a, count(b)
from R
group by a;

- Sort-based algorithm
- Steps:
  - Sort *R* on *a*
  - Now all tuples in a single group are contigous
  - Read tuples of *R (sorted)* one by one and compute the aggregates

Outer Joins

- Say: R FULL OUTER JOIN S, on R.a = S.a
- Need to keep track of which tuples of R “do not match” any tuples from S, and vice versa

- Hash-based, with a hash index on S:
  - For a tuple r in R, if the probe returns NULL, output r padded with NULLs
  - For each tuple *s* in *S*, maintain a Boolean variable (in the hash table) to track whether *s* was returned for any probes
  - At the end, go through the hash table, and look for *S* tuples that did not match anything

- Merge join can also be adapted in a similar way

Left Outer Hash Join

```
def open(self):
    self.H = build_hash_table(on="S", key="a")
    self.rScan = TableScan("R")
    self.rScan.open()
    self.matches = []
    self.r = None

def next(self):
    while True:
        if self.matches is not empty:
            s = self.matches.pop()
        return join(self.r, s)

        self.r = self.rScan.next()
        if self.r is EOF:
            return EOF

        self.matches = self.H.lookup(self.r.a)
        if self.matches is empty:
            return join(self.r, NULLs)

def close(self):
    self.rScan.close()
```

If "r" does not find a match in S, return it padded with an appropriate number of NULLs

Right Outer Hash Join

```
def open(self):
    self.H = build_hash_table(on="S", key="a")
    self.rScan = TableScan("R")
    self.rScan.open()
    self.matches = []
    self.r = None
    self.sNotMatched = set(all S tuples)
    self.phase = 1

def next(self):
    if phase == 1:
        while True:
            if self.matches is not empty:
                s = self.matches.pop()
                self.sNotMatched.remove(s)
            return join(self.r, s)

        self.r = self.rScan.next()
        if self.r is EOF:
            phase = 2

        self.matches = self.H.lookup(self.r.a)
```

Keep track of S tuples that were not returned by some R tuple

Once we are done with scanning R, move to Phase 2

Right Outer Hash Join

```
def next(self):
    if phase == 1:
        while True:
            if self.matches is not empty:
                s = self.matches.pop()
                self.sNotMatched.remove(s)
            return join(self.r, s)

        self.r = self.rScan.next()
        if self.r is EOF:
            phase = 2

        self.matches = self.H.lookup(self.r.a)


    if phase == 2:
        if self.sNotMatched is not empty:
            s = self.sNotMatched.pop()
            return join(NULLs, s)
        else:
            return EOF
```

Return all the non-matched S tuples one-by-one

Duplicate Elimination

select distinct a
from R ;

- Best done using sorting – Can also be done using hashing
- Steps:
  - Sort the relation *R*
  - Read tuples of *R* in sorted order
  - prev = null;
  - for each tuple *r* in *R (sorted)*
    - if *r* != *prev* then
      - Output *r*
      - *prev* = *r*
    - else
      - Skip *r*

Set operations

*(select * from R) union (select * from S) ;*
*(select * from R) intersect (select * from S) ;*
*(select * from R) union all (select * from S) ;*
*(select * from R) intersect all (select * from S) ;*

- Remember the rules about duplicates
- "union all": just append the tuples of *R and S*
- *"*union": append the tuples of *R and S, and do duplicate elimination*
- *"intersection"*: similar to **joins**
  - Find tuples of R and S that are identical on all attributes
  - Can use *hash-based or sort-based algorithm*

CMSC424: Database Design

Module: Query Processing and Optimization

External Memory

Instructor: Amol Deshpande
amol@umd.edu

Hash Join

- <u>Assume S is the smaller relation and fits in memory</u>

  read S in memory and build a hash index (hash table) on it

  for each tuple r in R

  use the hash index on S to find tuples such that S.a = r.a

- Cost: O(N_r + N_s + *O*) -- *where O is the number of output tuples*

- Why good ?
  - CPU cost is much better

Hash Join

- <u>Case 2: Smaller relation *(S)* doesn't fit in memory</u>
- Two "phases"
- Phase 1:
  - Read the relation *R* block by block and partition it using a hash function, *h1(a)*
    - Create one partition for each possible value of *h1(a)*
  - Write the partitions to disk
    - R gets partitioned into R1, R2, …, Rk
  - Similarly, read and partition S, and write partitions S1, S2, …, Sk to disk
  - Only requirement:
    - Each S partition fits in memory

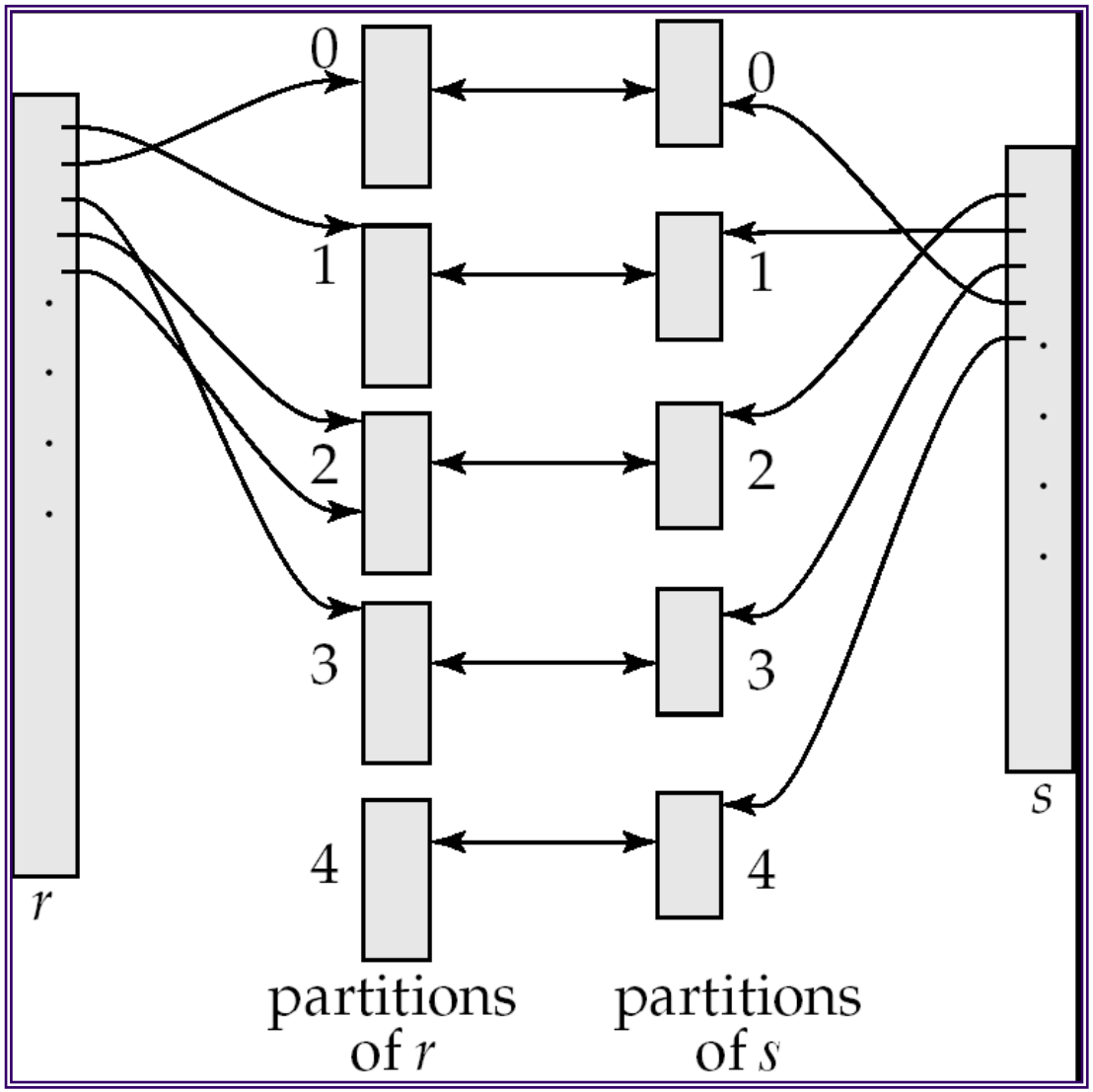
Hash Join

- <u>Case 2: Smaller relation *(S)* doesn't fit in memory</u>
- Two "phases"
- Phase 2:
  - Read S1 into memory, and bulid a hash index on it (S1 fits in memory)
    - Using a different hash function, $h_2(a)$
  - Read R1 block by block, and use the hash index to find matches.
  - Repeat for S2, R2, and so on.

Hash Join

- <u>Case 2: Smaller relation *(S)* doesn't fit in memory</u>
- Two "phases":
- Phase 1:
  - Partition the relations using one hash function, $h_1(a)$
- Phase 2:
  - Read S_i into memory, and bulid a hash index on it (S_i fits in memory)
  - Read R_i block by block, and use the hash index to find matches.
- Cost ?
  - We read and write the two relations thrice (instead of once)

Hash Join



Hash Join: Issues

- How to guarantee that the partitions of *S* all fit in memory ?
  - Say S = 10000 blocks, Memory = M = 100 blocks
  - Use a hash function that hashes to 100 different values ?
    - Eg. *h1(a) = a % 100 ?*
  - Problem: Impossible to guarantee uniform split
    - Some partitions will be larger than 100 blocks, some will be smaller
  - Use a hash function that hashes to *100*f* different values
    - *f* is called fudge factor, typically around 1.2
    - So we may consider *h1(a) = a % 120.*
    - This is okay IF *a* is uniformly distributed
  - What if the hash function turns out to be bad ?
    - Repartition using a different hash function (at run time)

Sorting

- Commonly required for many operations
  - Duplicate elimination, group by's, sort-merge join
  - Queries may have ASC or DSC in the query
- One option:
  - Read the lowest level of the index
    - May be enough in many cases
  - But if relation not sorted, this leads to too many random accesses
- If relation small enough…
  - Read in memory, use quick sort (qsort() in C)
- What if relation too large to fit in memory ?
  - External sort-merge

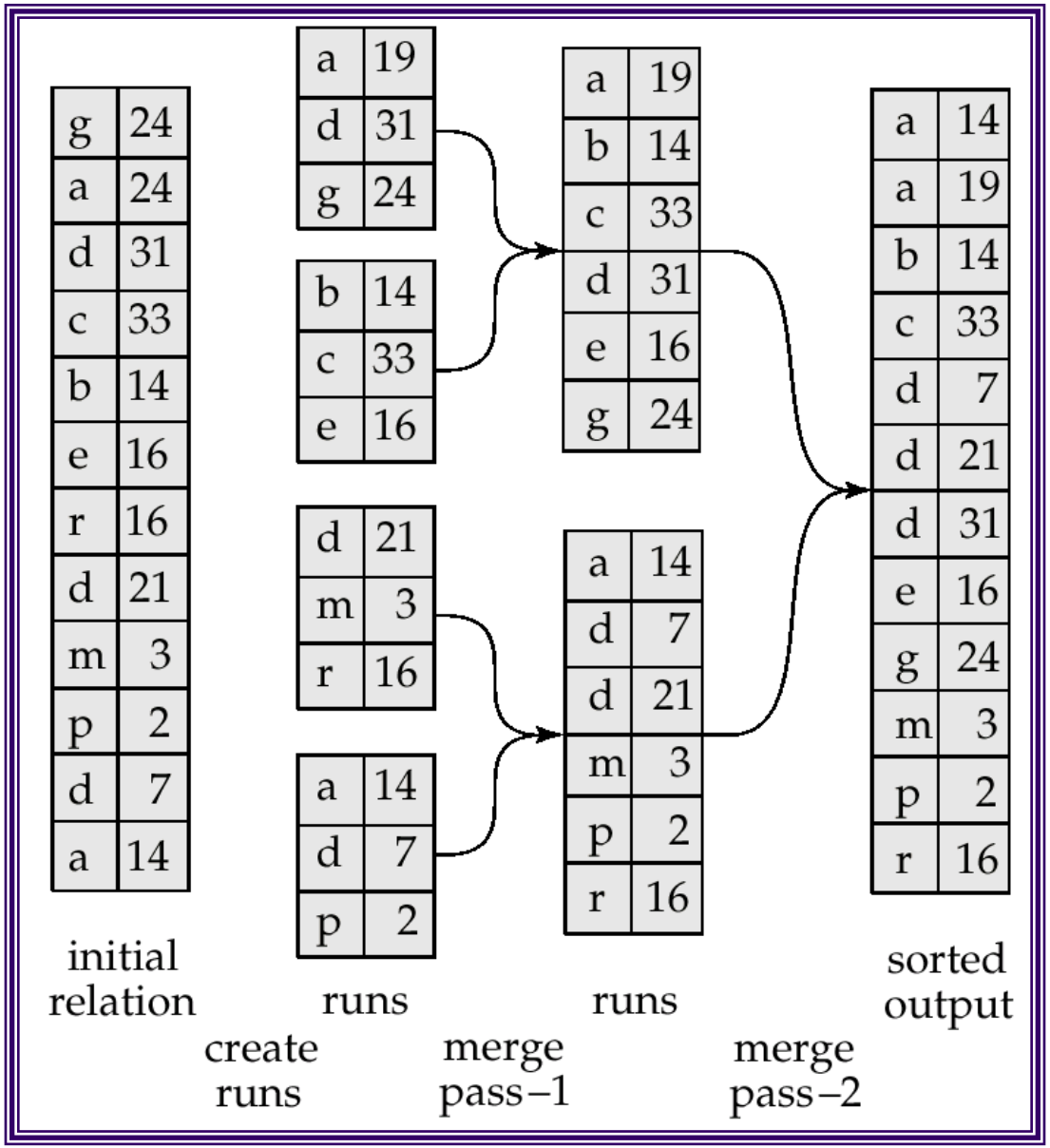
External sort-merge

- Divide and Conquer !!
- Let *M* denote the memory size (in blocks)
- Phase 1:
  - Read first M blocks of relation, sort, and write it to disk
  - Read the next M blocks, sort, and write to disk …
  - Say we have to do this “N” times
  - Result: *N* sorted runs of size *M* blocks each
- Phase 2:
  - Merge the *N* runs (*N-way merge)*
  - Can do it in one shot if $N < M$

External sort-merge

- Phase 1:
  - Create *sorted runs of size M* each
  - Result: *N* sorted runs of size *M* blocks each
- Phase 2:
  - Merge the *N* runs (*N-way merge)*
  - Can do it in one shot if $N < M$
- What if $N > M$ *?*
  - Do it recursively
  - Not expected to happen
  - If M = 1000 blocks = 4MB (assuming blocks of 4KB each)
    - Can sort: 4000MB = 4GB of data

Example: External Sorting Using Sort-Merge



CMSC424: Database Design

Module: Query Processing and Optimization

Query Optimization

Instructor: Amol Deshpande
amol@umd.edu

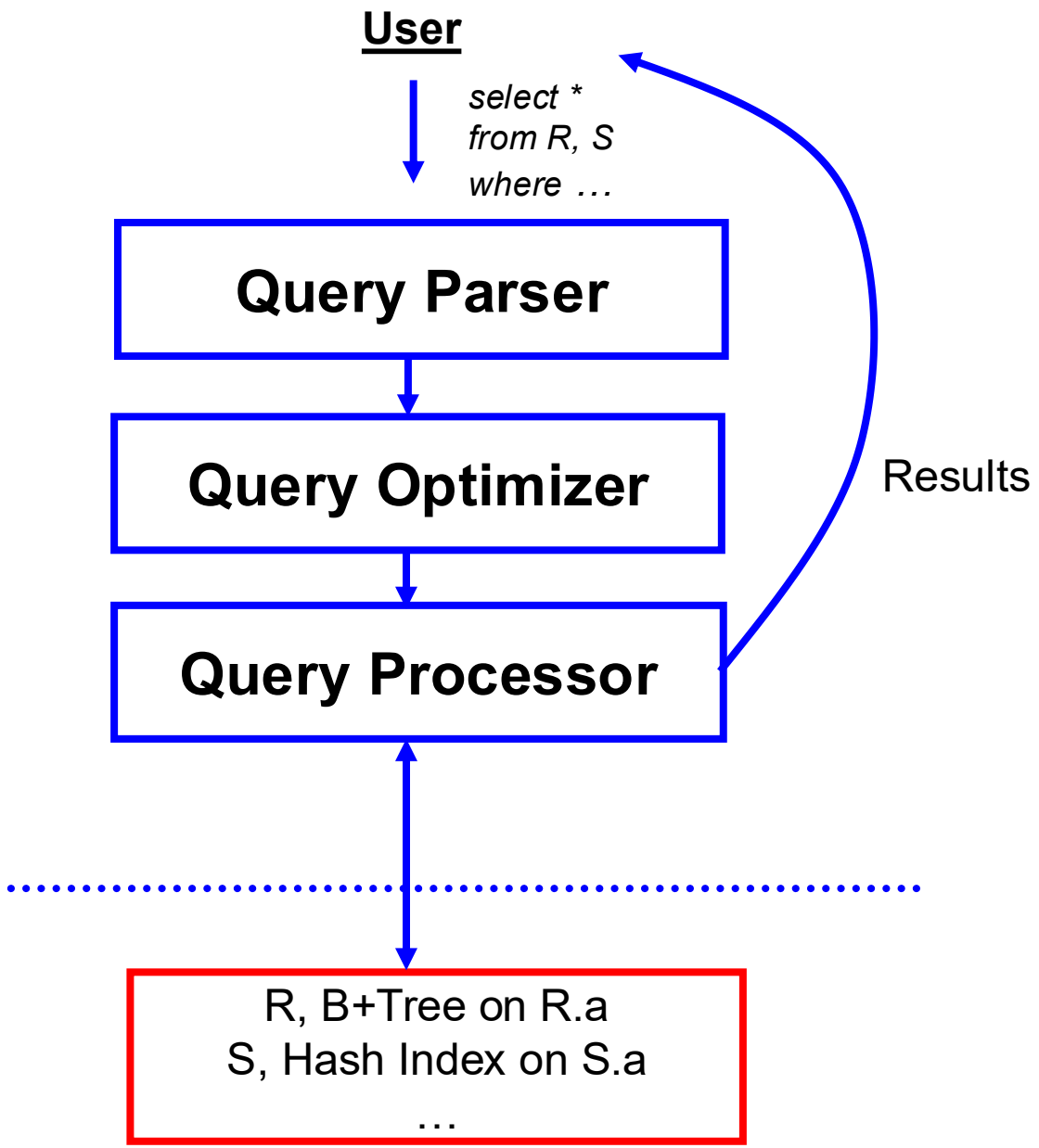
Recap: Query Processing

- Many, many ways to implement the relational operations
  - Numerous more used in practice
  - Especially in data warehouses which handle TBs (even PBs) of data
- However, consider how complex SQL is and how much you can do
  - Compared to that, this isn't much
- Most of it is very nicely modular
  - Can plug in new operators quite easily
  - PostgreSQL query processing codebase easy to read and modify
- Having so many operators does complicate the codebase and the query optimizer though
  - But needed for performance

Query Optimization: Overview

- Key topics:
  - Why query optimization is so important?
  - Key steps in query optimization
  - High-level concepts

Getting Deeper into Query Processing



Resolve the references,
Syntax errors etc.
Converts the query to an internal format
relational algebra like

Find the *best* way to evaluate the query
Which index to use ?
What join method to use ?
…

Read the data from the files
Do the query processing
joins, selections, aggregates
…

Query Optimization

- Why ?
  - Many different ways of executing a given query
  - Huge differences in cost
- Example:
  - select * from person where ssn = “123”
  - Size of *person* = 1GB
  - Sequential Scan:
    - Takes 1GB / (20MB/s) = 50s
  - Use an index on SSN (assuming one exists):
    - Approx 4 Random I/Os = 40ms

Query Optimization

- Many choices
  - Using indexes or not, which join method (hash, vs merge, vs NL)
  - What join order ?
    - Given a join query on R, S, T, should I join R with S first, or S with T first ?
- This is an optimization problem
  - Similar to say *traveling salesman problem*
  - Number of different choices is very very large
  - Step 1: Figuring out the *solution space*
  - Step 2: Finding algorithms/heuristics to search through the solution space

Query Optimization: Goal

- Find the best (or a good enough) execution plan
- Execution plans = Evaluation expressions annotated with the methods used

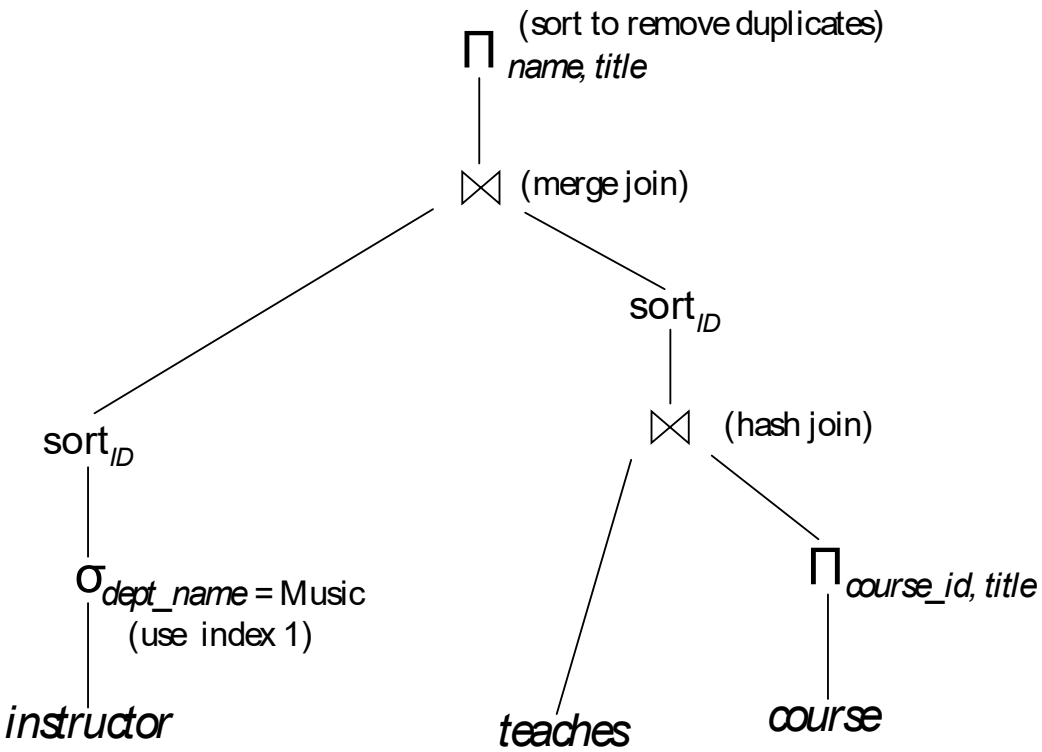


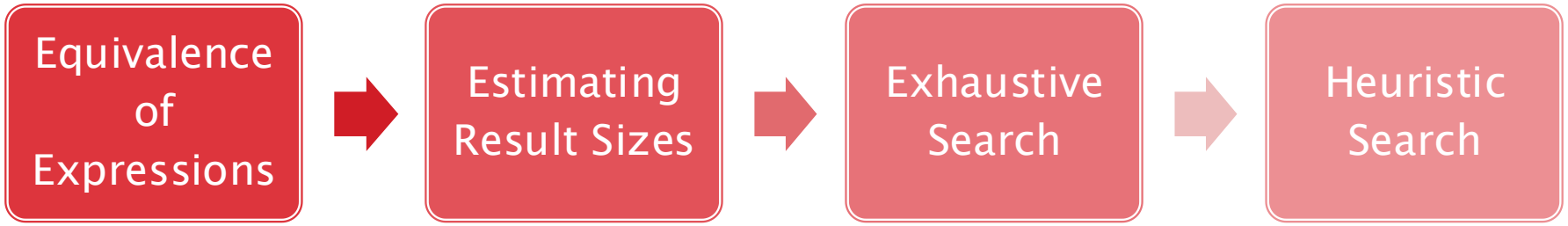
Figure 16.2 An evaluation plan.

Query Optimization

- Steps:
  - Generate all possible execution plans for the query
  - Figure out the cost for each of them
  - Choose the best

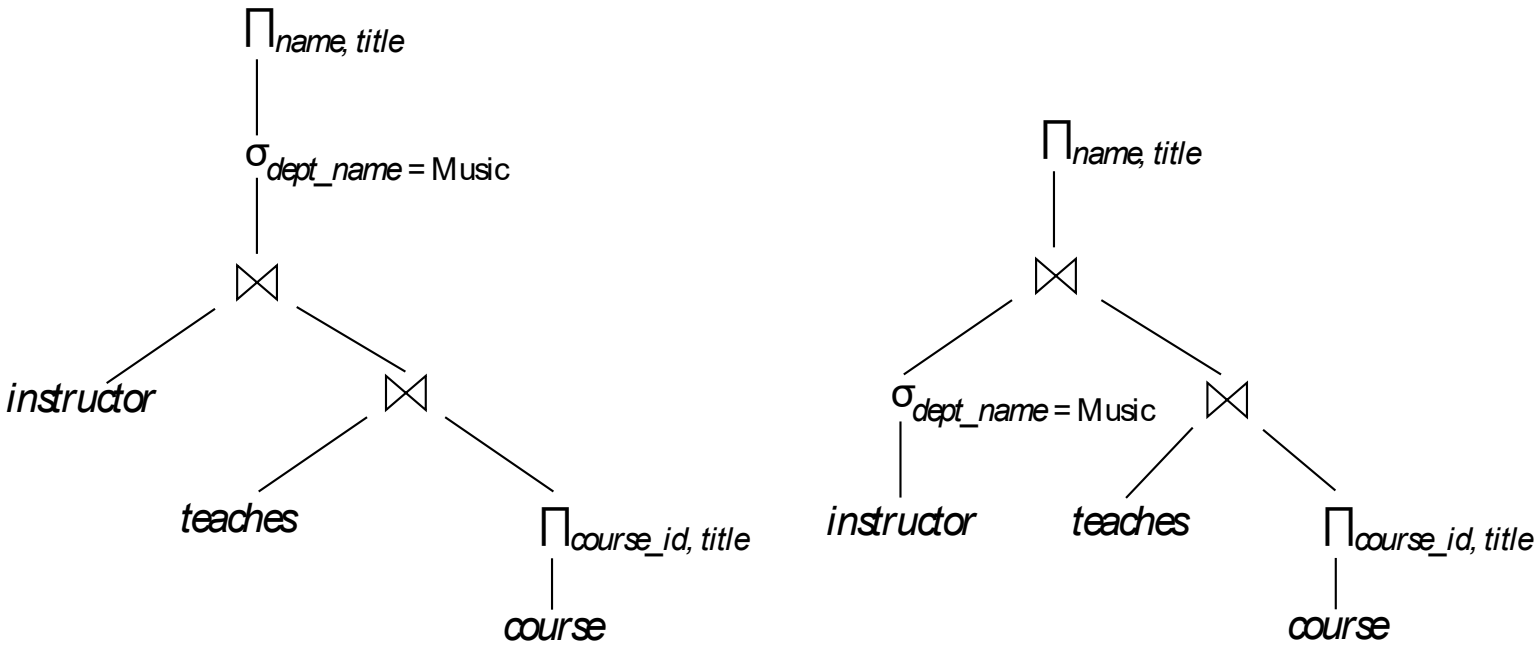
- Not done exactly as listed above
  - Too many different execution plans for that
  - Typically interleave all of these into a single efficient search algorithm

Query Optimization Outline



Equivalence of Expressions

- Equivalent relational expressions
  - Drawn as a tree
  - List the operations and the order



(a) Initial expression tree

(b) Transformed expression tree

Figure 16.1 Equivalent expressions.

Equivalence of Expressions

- Two relational expressions equivalent iff:
  - Their result is identical on all legal databases
- Equivalence rules:
  - Allow replacing one expression with another
- Examples:

1. $\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$

2. Selections are commutative

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

Equivalence Rules

- Examples:

  3. $\Pi_{L_1}(\Pi_{L_2}(\ldots(\Pi_{Ln}(E))\ldots)) = \Pi_{L_1}(E)$

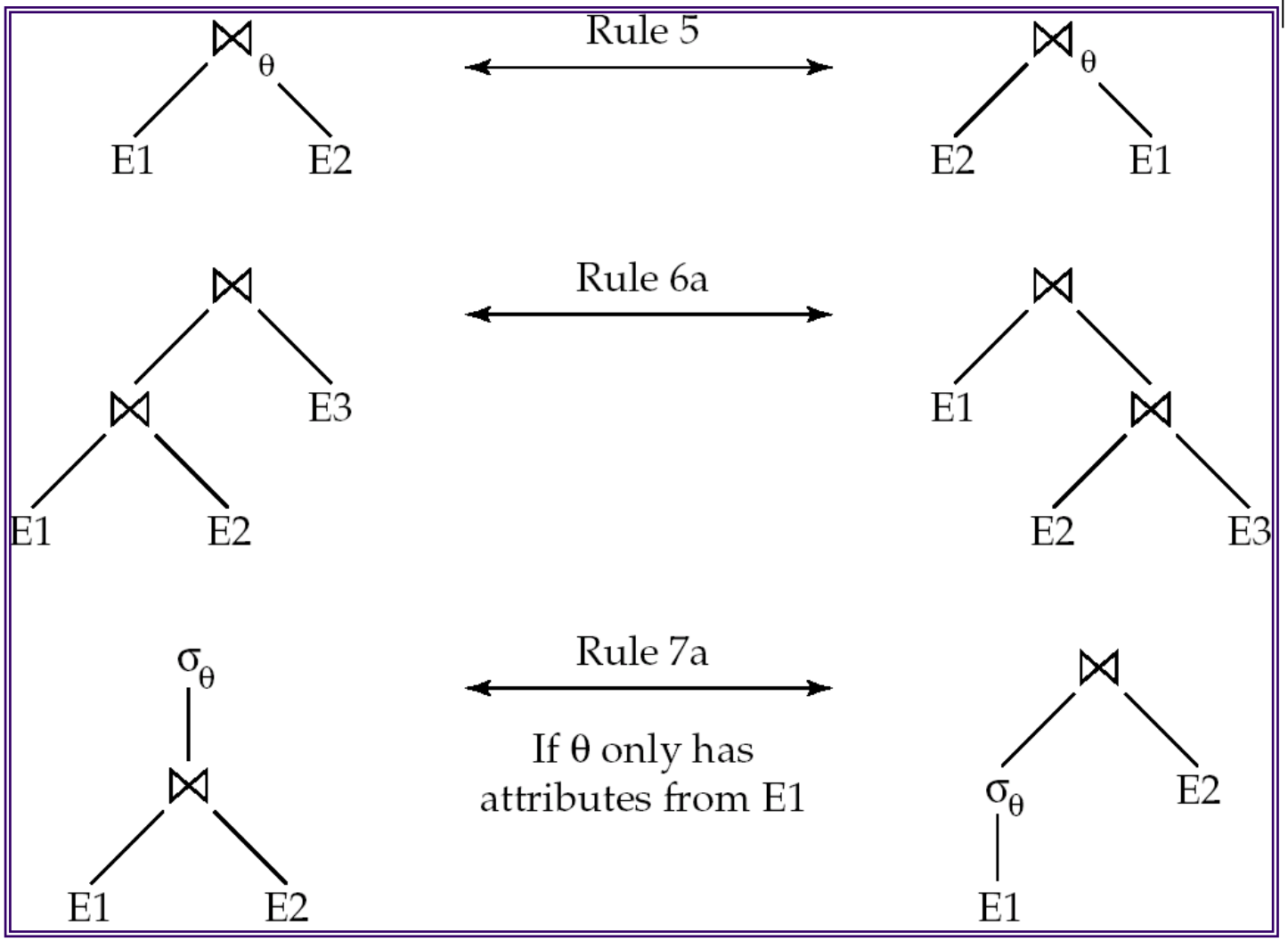
  5. $E_1 \bowtie_\theta E_2 = E_2 \bowtie_\theta E_1$

  7(a). If θ_0 *only involves attributes from* E_1

  $$\sigma_{\theta 0}(E_1 \bowtie_\theta E_2) = (\sigma_{\theta 0}(E_1)) \bowtie_\theta E_2$$

- And so on…
  - Many rules of this type

Pictorial Depiction



More Equivalence Rules

- The set operations union and intersection are commutative and associative

$$E_1 \cup E_2 \equiv E_2 \cup E_1$$
$$E_1 \cap E_2 \equiv E_2 \cap E_1$$
$$(E_1 \cup E_2) \cup E_3 \equiv E_1 \cup (E_2 \cup E_3)$$
$$(E_1 \cap E_2) \cap E_3 \equiv E_1 \cap (E_2 \cap E_3)$$

- But not set difference

$$E_1 - E_2 \not\equiv E_2 - E_1$$

More Equivalence Rules

- Outerjoins are more involved

$$E_1 ⟗ E_2 \equiv E_2 ⟗ E_1$$

$$E_1 ⟕ E_2 \equiv E_2 ⟖ E_1$$

- Outerjoins are not associative

$$(r ⟕ s) ⟕ t \not\equiv r ⟕ (s ⟕ t)$$

  - e.g. with r(A,B) = {(1,1), s(B,C) = { (1,1)}, t(A,C) = { }

Equivalence of Expressions

- The rules give us a way to enumerate all equivalent expressions
  - Note that the expressions don't contain physical access methods, join methods etc…
- Simple Algorithm:
  - Start with the original expression
  - Apply all possible applicable rules to get a new set of expressions
  - Repeat with this new set of expressions
  - Till no new expressions are generated

Equivalence of Expressions

- Works, but is not feasible
- Consider a simple case:
  - $R1 \bowtie (R2 \bowtie (R3 \bowtie (\ldots \bowtie Rn)))\ldots.)$

- Just join commutativity and associativity will give us:
  - At least:
    - n^2 * 2^n
  - At worst:
    - n! * 2^n
- Typically the process of enumeration is combined with the search process

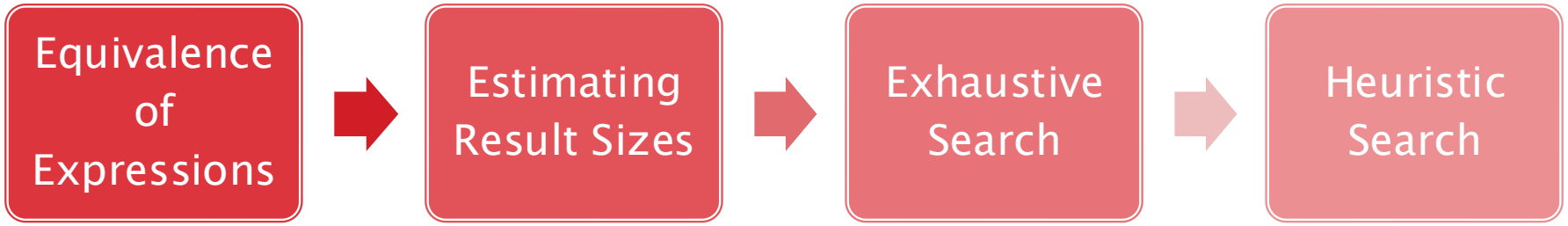
Evaluation Plans

- We still need to choose the join methods etc..
  - Option 1: Choose for each operation separately
    - Usually okay, but sometimes the operators interact
    - Consider joining three relations on the same attribute:
      - $R1 \bowtie_a (R2 \bowtie_a R3)$
    - Best option for R2 join R3 might be hash-join
      - But if *R1* is sorted on *a,* then *sort-merge join* is preferable
      - Because it produces the result in sorted order by *a*
- Also, we need to decide whether to use pipelining or materialization
- Such issues are typically taken into account when doing the optimization

Query Optimization

- Steps:
  - Generate all possible execution plans for the query
    - First generate all equivalent expressions
    - Then consider all annotations for the operations
  - Figure out the cost for each of them
    - Compute cost for each operation
      - Using the formulas discussed before
      - One problem: How do we know the number of result tuples for, say, $\sigma_{balance<2500}(account)$
    - Add them !
  - Choose the best

Query Optimization Outline



Cost estimation

- Computing operator costs requires information like:
  - Primary key ?
  - Sorted or not, which attribute
    - So we can decide whether need to sort again
  - How many tuples in the relation, how many blocks ?
  - How many tuples match a predicate like “age > 40” ?
    - E.g. Need to know how many index pages need to be read
  - Intermediate result sizes
    - E.g. (R JOIN S) is input to another join operation – need to know if it fits in memory
  - And so on…

Cost estimation

- Some information is static and is maintained in the metadata
  - Primary key ?
  - Sorted or not, which attribute
    - So we can decide whether need to sort again
  - How many tuples in the relation, how many blocks ?
- Typically kept in some tables in the database
  - “all_tab_columns” in Oracle
- Most systems have commands for updating them

Cost estimation

- However, others need to be estimated somehow
  - How many tuples match a predicate like “age > 40” ?
    - E.g. Need to know how many index pages need to be read
  - Intermediate result sizes
- The problem variously called:
  - “intermediate result size estimation”
  - “selectivity estimation”

- Very important to estimate reasonably well
  - e.g. consider “select * from R where zipcode = 20742”
  - We estimate that there are 10 matches, and choose to use a secondary index (remember: random I/Os)
  - Turns out there are 10000 matches
  - Using a secondary index very bad idea
  - Optimizer also often choose Nested-loop joins if one relation very small… underestimation can result in very bad

Selectivity Estimation

- Basic idea:
  - Maintain some information about the tables
    - More information → more accurate estimation
    - More information → higher storage cost, higher update cost
  - Make uniformity and randomness assumptions to fill in the gaps

- Example:
  - For a relation “people”, we keep:
    - Total number of tuples = 100,000
    - Distinct “zipcode” values that appear in it = 100
  - Given a query: “zipcode = 20742”
    - We estimated the number of matching tuples as: 100,000/100 = 1000
  - What if I wanted more accurate information ?
    - Keep better statistics/summaries…

Basic statistics

- Basic information stored for all relations
  - n_r: number of tuples in a relation *r.*
  - b_r: number of blocks containing tuples of *r.*
  - l_r: size of a tuple of *r.*
  - f_r: blocking factor of *r* — i.e., the number of tuples of *r* that fit into one block.
  - *V(A, r):* number of distinct values that appear in *r* for attribute *A;* same as the size of $\prod_A(r)$.
  - *MAX(A, r):* th maximum value of *A* that appears in *r*
  - *MIN(A, r)*
  - If tuples of *r* are stored together physically in a file, then:

$$b_r = \left\lceil \frac{n_r}{f_r} \right\rceil$$

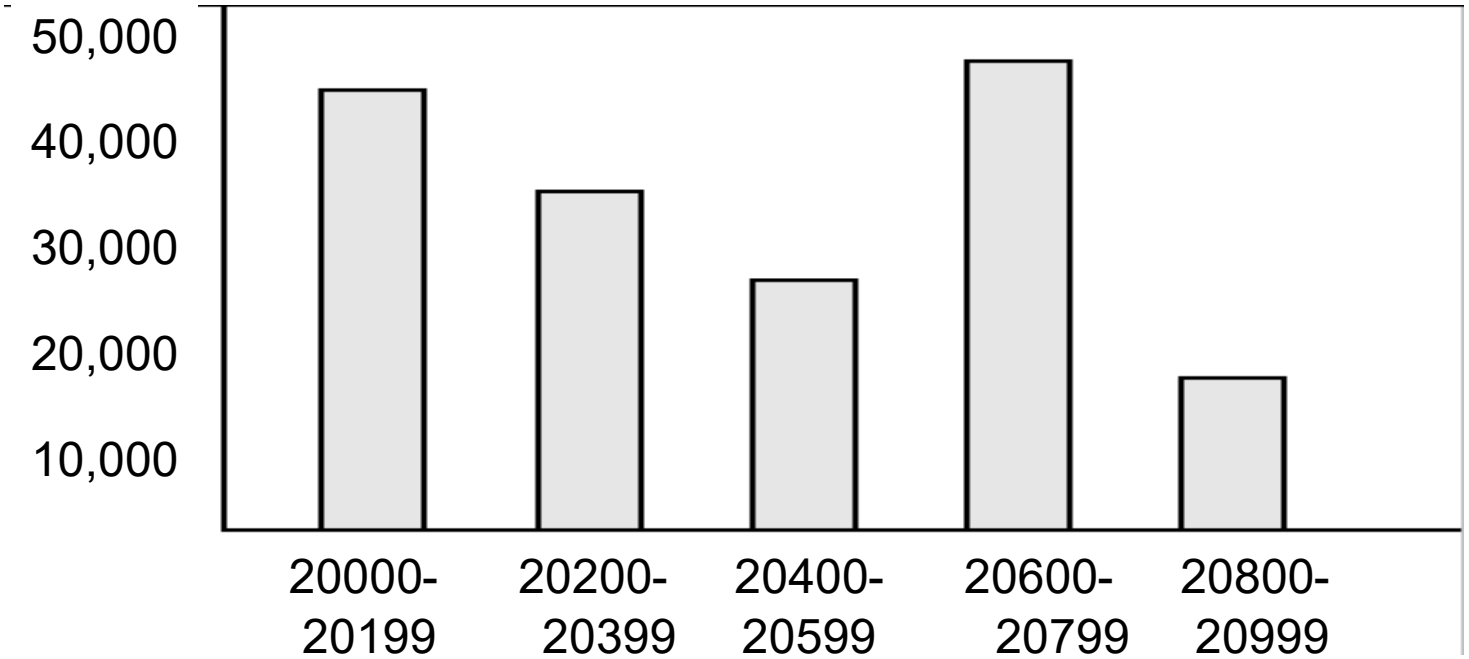
Examples

- Consider a range query: $x < R.a < y$
  - *Let Max(a, R) = maximum value of a in R*
  - *Let Min(a, R) = minimum value of a in R*
  - *Then: fraction of tuples that satisfy = (y – x) / (Max – Min)*
    - *Assuming all tuples are distributed uniformly and randomly*
    - *If y > Max or x < Min → adjust accordingly*

  - Better summary statistics (like histograms) can help with refining these estimates

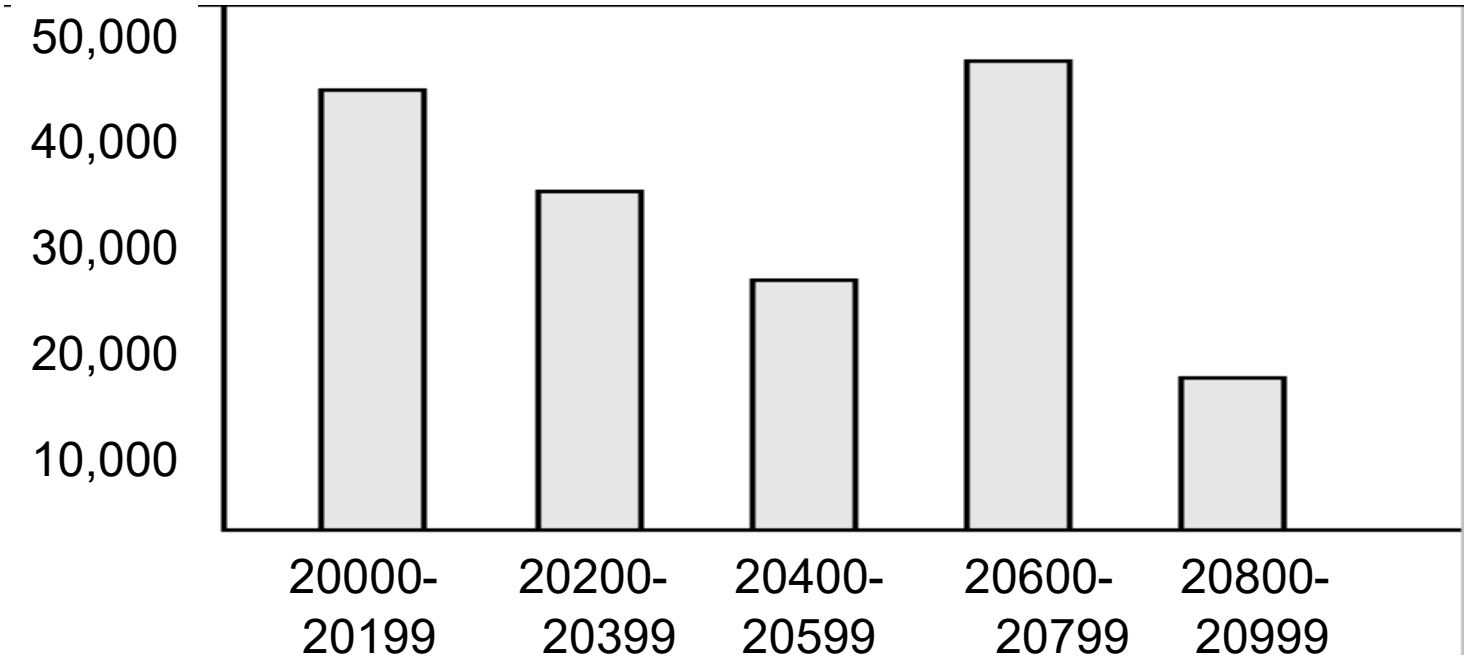
Histograms

- A condensed, approximate version of the “frequency distribution”
  - Divide the range of the attribute value in “buckets”
  - For each bucket, keep the total count
  - Assume uniformity within a bucket



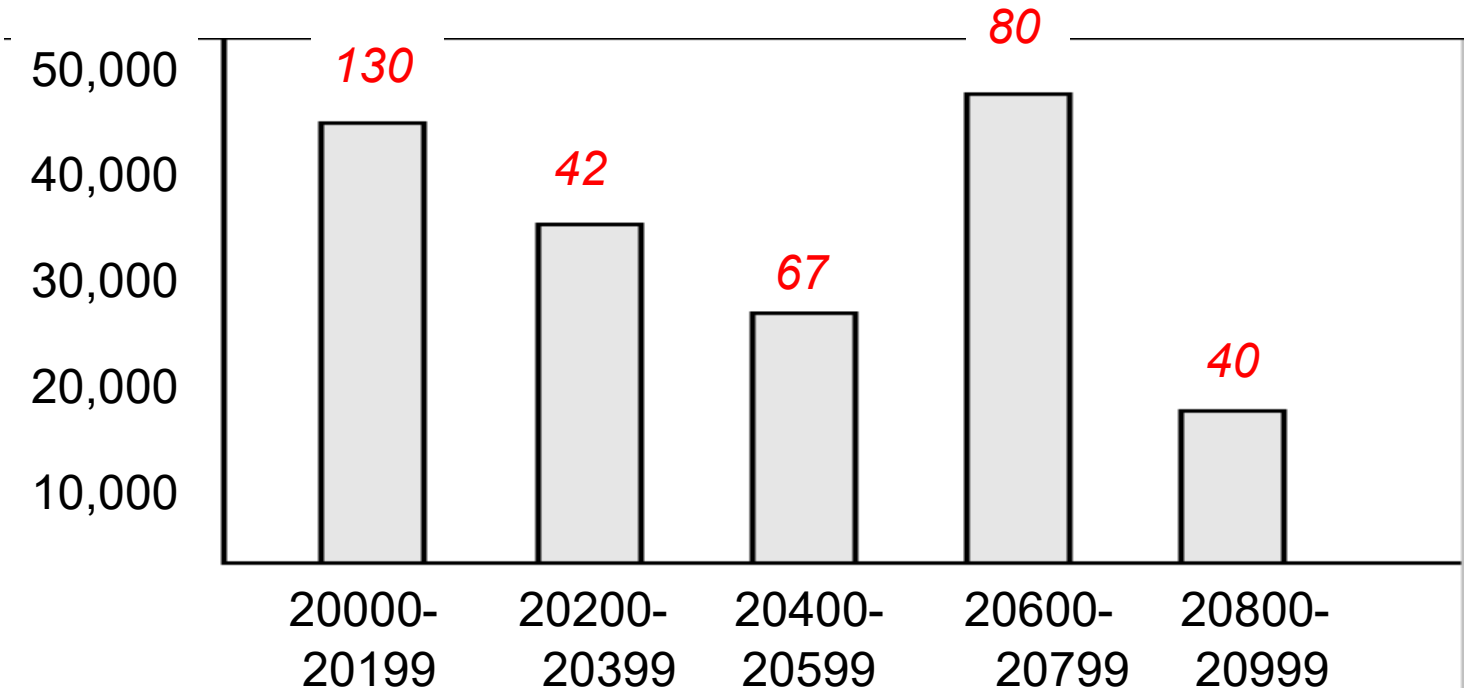
Histograms

- Given a query: zipcode = “ 20742”
  - Find the bucket (Number 3)
  - Say the associated cound = 45000
  - Assume uniform distribution within the bucket: 45,000/200 = 225



Histograms

- What if the ranges are typically not full ?
  - ie., only a few of the zipcodes are actually in use ?
- With each bucket, also keep the number of zipcodes that are valid
- Now the estimate would be: 45,000/80 = 562.50
- More Information → Better estimation



Histograms

- Very widely used in practice
  - One-dimensional histograms kept on almost all columns *of interest*
    - ie., the columns that are commonly referenced in queries
  - Sometimes: multi-dimensional histograms also make sense
    - Less commonly used as of now
- Two common types of histograms:
  - Equi-depth
    - The attribute value range partitioned such that each bucket contains about the same number of tuples
  - Equi-width
    - The attribute value range partitioned in equal-sized buckets
  - VOptimal histograms
    - No such restrictions
    - More accurate, but harder to use or update

Example: Joins

- R JOIN S: R.a = S.a
  - |R| = 10,000; |S| = 5000

- CASE 1: *a* is key for S
  - *Each tuple of R joins with exactly one tuple of S*
  - So: |R JOIN S| = |R| = 10,000
  - Assumption: Referential integrity holds

  - What if there is a selection on R or S
    - Adjust accordingly
    - Say: S.b = 100, with selectivity 0.1
    - THEN: |R JOIN S| = |R| * 0.1 = 100

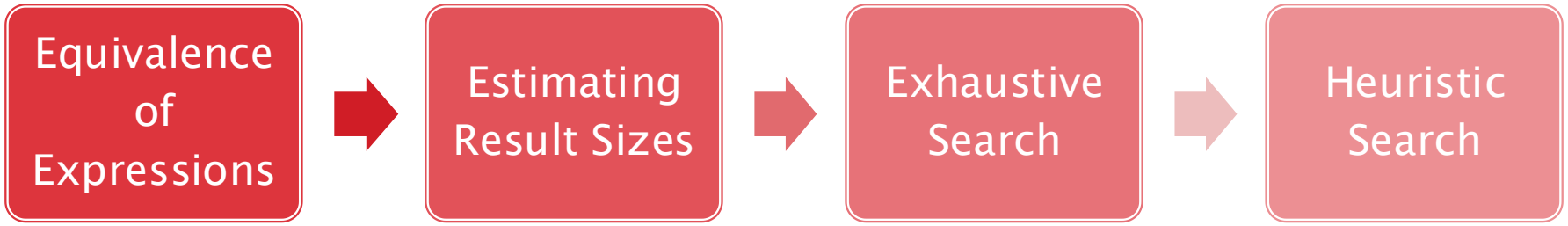
- CASE 2: *a* is key for R
  - Similar

Joins

- R JOIN S: R.a = S.a
  - |R| = 10,000; |S| = 5000

- CASE 3: *a* is not a key for either
  - Reason with the distributions on *a*
  - Say: the domain of *a: V(A, R)* = 100 (the number of distinct values *a* can take)
  - THEN, *assuming uniformity*
    - For each value of *a*
      - We have 10,000/100 = 100 tuples of R with that value of a
      - We have 5000/100 = 50 tuples of S with that value of a
      - All of these will join with each other, and produce 100 *50 = 5000
    - So total number of results in the join:
      - 5000 * 100 = 500000
  - We can improve the accuracy if we know the distributions on *a* better
    - Say using a histogram

Query Optimization Outline



Query Optimization

- Steps:
  - Generate all possible execution plans for the query
    - First generate all equivalent expressions
    - Then consider all annotations for the operations
  - Figure out the cost for each of them
    - Compute cost for each operation
      - Using the formulas discussed before
      - One problem: How do we know the number of result tuples for, say, $\sigma_{balance<2500}(account)$
    - Add them !
  - Choose the best

Optimization Algorithms

- Two types:
  - Exhaustive: That attempt to find the best plan
  - Heuristical: That are simpler, but are not guaranteed to find the optimal plan

- Consider a simple case
  - Join of the relations *R1, …, Rn*
  - No selections, no projections

- Still very large plan space

Searching for the best plan

- <u>Option 1:</u>
  - Enumerate all equivalent expressions for the original query expression
    - Using the rules outlined earlier
  - Estimate cost for each and choose the lowest

- Too expensive !
  - Consider finding the best join-order for $r_1 \bowtie r_2 \bowtie \ldots r_n$.
  - There are $(2(n-1))!/(n-1)!$ different join orders for above expression. With $n = 7$, the number is 665280, with $n = 10$, the number is greater than 176 billion!

Searching for the best plan

- <u>Option 2:</u>
  - Dynamic programming
    - There is too much commonality between the plans
    - Also, costs are additive
      - Caveat: Sort orders (also called “interesting orders”)
  - Reduces the cost down to O(n3^n) or O(n2^n) in most cases
    - Interesting orders increase this a little bit
  - Considered acceptable
    - Typically n < 10.
  - Switch to heuristic if not acceptable

Dynamic Programming Algo.

- Join R1, R2, R3, R4, R5

$R1 \bowtie R2 \bowtie R3$

Options:
1. Join R1R2 with R3 using HJ
   cost = 100 + cost of this join
2. Join R1R2 with R3 using SMJ
   cost = 100 + cost of this join
3. Join R1R3 with R2 using HJ
   cost = 300 + cost of this join

…

$R1 \bowtie R2$	$R1 \bowtie R3$	$R1 \bowtie R4$	$R4 \bowtie R5$
cost: 100	cost: 300	….	cost: 300
plan: HJ	plan: SMJ		plan: HJ

R1 R2 R3 R4 R5

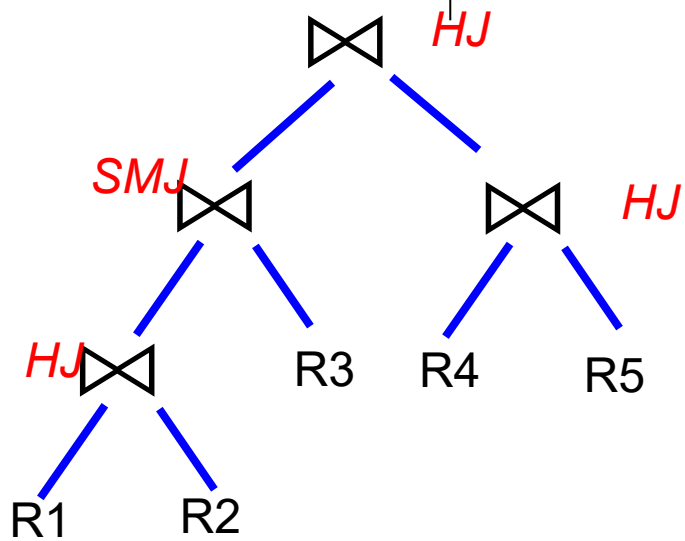
R1 ⋈ R2 ⋈ R3 ⋈ R4 ⋈ R5
cost: 1200
plan: *HJ(R1R2R3, R4R5)*

R1 ⋈ R2 ⋈ R3 ⋈ R4
cost: 700
plan: *HJ(R1R2R3, R4)*

….

R1 ⋈ R2 ⋈ R3
cost: 400
plan: *SMJ(R1R2, R3)*

….



R1 ⋈ R2
cost: 100
plan: HJ

R1 ⋈ R3
cost: 300
plan: SMJ

R1 ⋈ R4
….

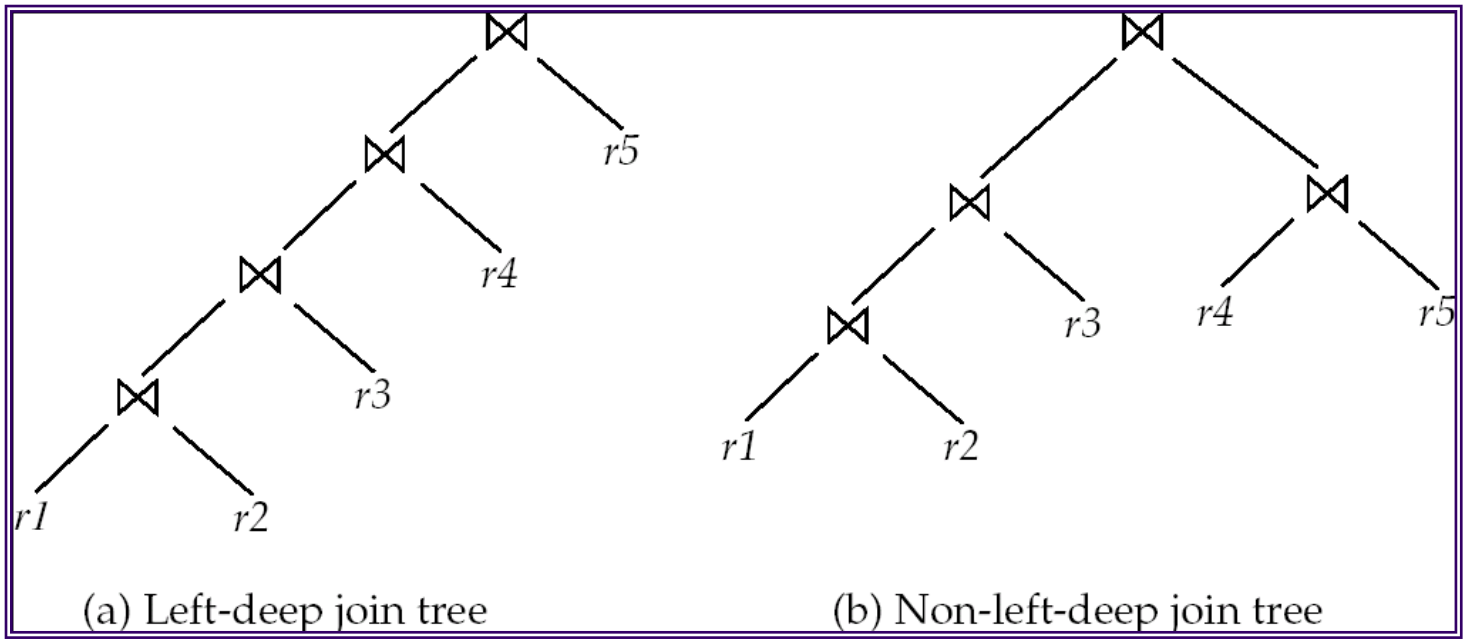
….

R4 ⋈ R5
cost: 300
plan: HJ

R1 R2 R3 R4 R5

Left Deep Join Trees

- In **left-deep join trees,** the right-hand-side input for each join is a relation, not the result of an intermediate join
- Early systems only considered these types of plans
  - Easier to pipeline



(a) Left-deep join tree (b) Non-left-deep join tree

Heuristic Optimization

- Dynamic programming is expensive
- Use *heuristics* to reduce the number of choices
- Typically rule-based:
  - Perform selection early (reduces the number of tuples)
  - Perform projection early (reduces the number of attributes)
  - Perform most restrictive selection and join operations before other similar operations.
- Some systems use only heuristics, others combine heuristics with partial cost-based optimization.

Summary

- Integral component of query processing
  - Why ?
- One of the most complex pieces of code in a database system
- Active area of research
  - E.g. MongoDB Query Optimization ?
  - What if you don’t know anything about the statistics
  - Better statistics
  - Etc …

CMSC424: Database Design

Module: Query Processing and Optimization

Parallel/Distributed Architectures; Data Replication; Sharding; Failures

Instructor: Amol Deshpande
amol@umd.edu

Parallel and Distributed Architectures

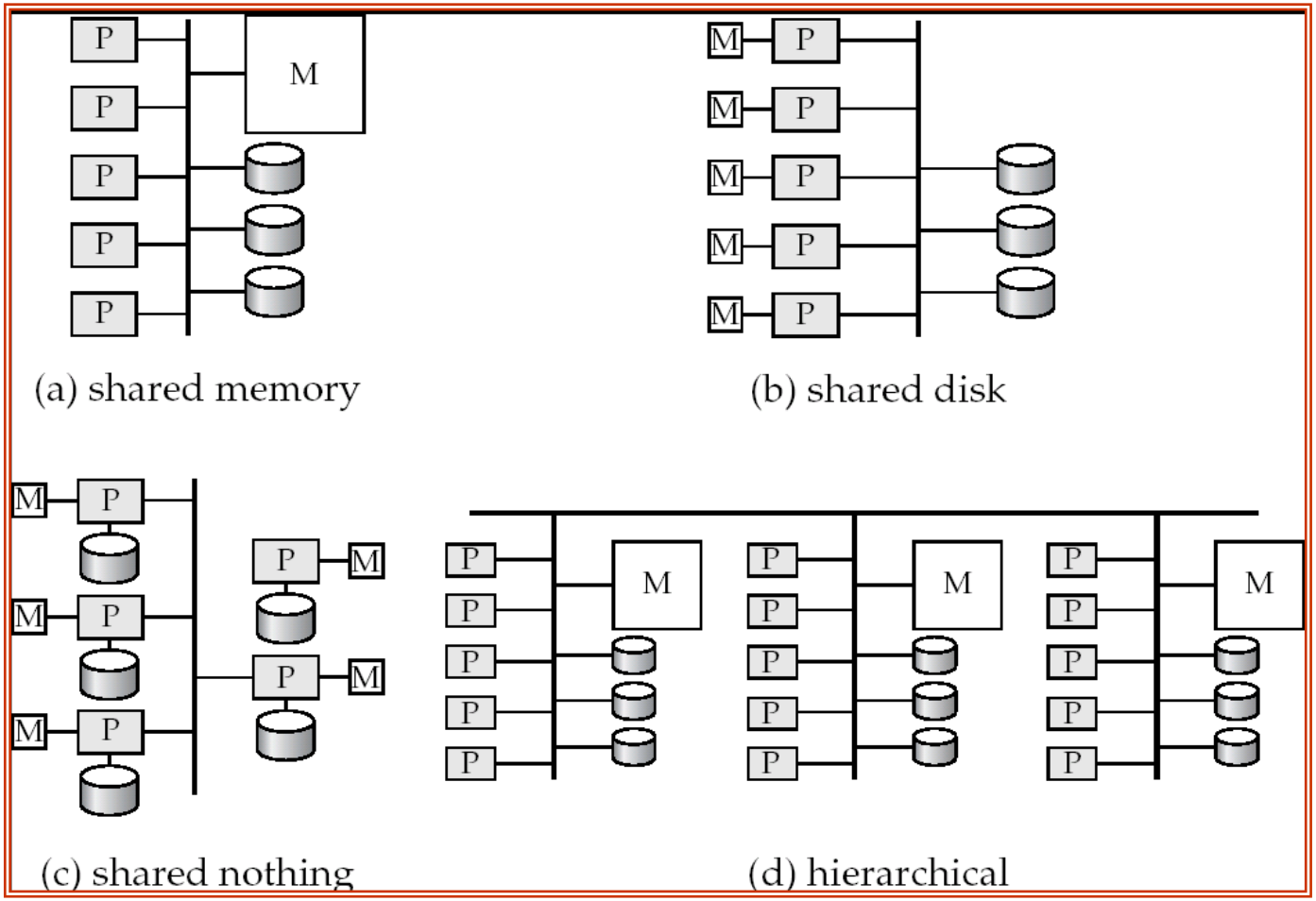
- Ability to scale “up” a computer is limited ➔ Use many computers together
  - Called cluster or network of computers (and today, just a “data center”)

- Also need to ”meet” where the users are
  - To minimize interactive latencies (e.g., social networks)

- Has made parallel and distributed architectures very common today

Parallel Architectures

- Shared-nothing vs. shared-memory vs. shared-disk



Parallel Architectures

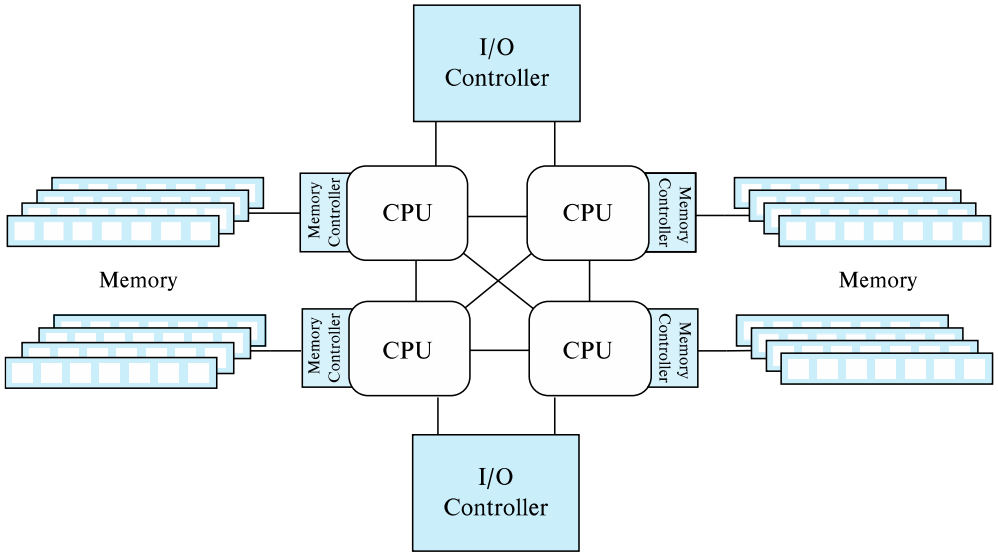


Figure 20.6 Architecture of a modern shared-memory system.

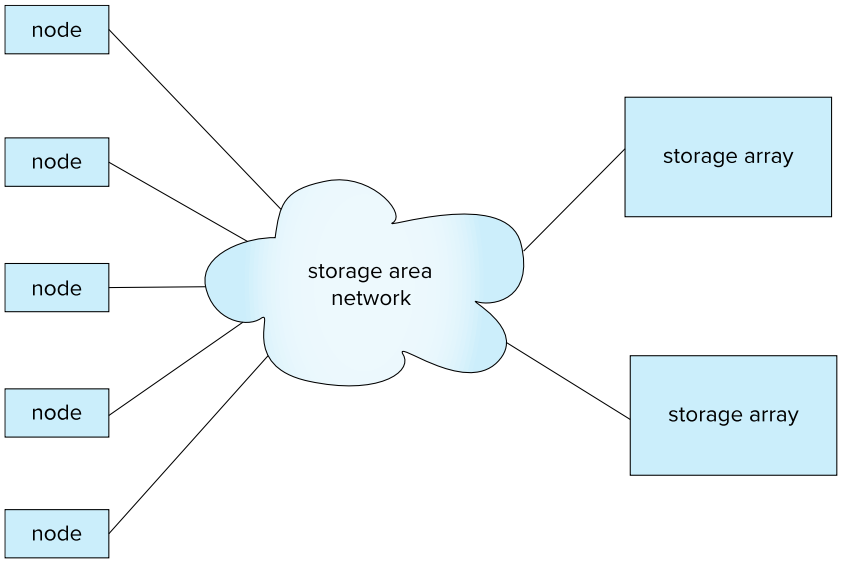


Figure 20.8 Storage-area network.

Parallel Architectures

	Shared Memory	Shared Disk	Shared Nothing
Communication between processors	Extremely fast	Disk interconnect is very fast	Over a LAN, so slowest
Scalability ?	Not beyond 32 or 64 or so (memory bus is the bottleneck)	Not very scalable (disk interconnect is the bottleneck)	Very very scalable
Notes	Cache-coherency an issue	Transactions complicated; natural fault-tolerance.	Distributed transactions are complicated (deadlock detection etc);
Main use	Low degrees of parallelism	Not used very often	Everywhere

Parallel Systems

- A **coarse-grain parallel** machine → a small number of powerful processors
- A **massively parallel** or **fine grain parallel** machine → thousands of smaller processors.
- We see a variety of mixes of these today, especially with the rise of multi-core machines

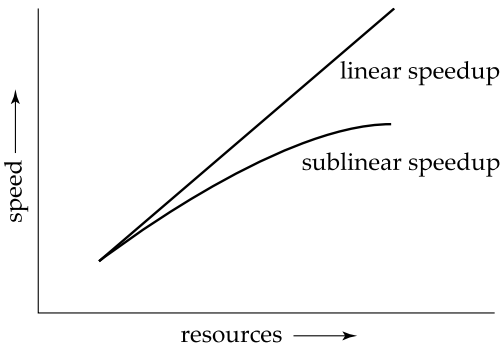
- Two main performance measures:
  - **throughput** --- the number of tasks that can be completed in a given time interval
  - **response time** --- the amount of time it takes to complete a single task from the time it is submitted

Speed-Up and Scale-Up

- **Speedup**: a fixed-sized problem executing on a small system is given to a system which is *N*-times larger.
  - Measured by:

    $$speedup = \frac{small\ system\ elapsed\ time}{large\ system\ elapsed\ time}$$

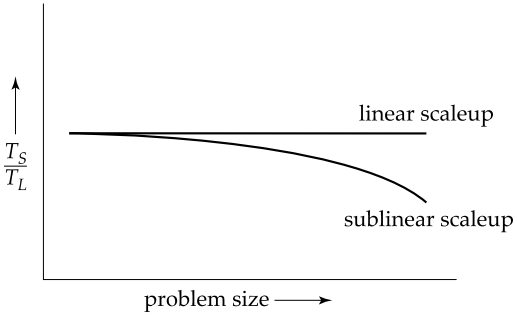
  - Speedup is **linear** if equation equals N.



- **Scaleup**: increase the size of both the problem and the system
  - *N*-times larger system used to perform *N*-times larger job
  - Measured by:

    $$scaleup = \frac{small\ system\ small\ problem\ elapsed\ time}{big\ system\ big\ problem\ elapsed\ time}$$

  - Scale up is **linear** if equation equals 1.



Factors Limiting Speedup and Scaleup

- **Sequential computation:** Some parts may not be parallalelizable
  - **Amdahl's Law**: If "p" is the fraction of the task that can be parallelized, then the best speedup you can get is: $\frac{1}{(1-p)+(p/n)}$
  - If "p" is 0.9, the best speedup is 10

- **Startup costs**: Cost of starting up multiple processes may dominate computation time, if the degree of parallelism is high.

- **Interference**: Processes accessing shared resources (e.g., system bus, disks, or locks) compete with each other, thus spending time waiting on other processes, rather than performing useful work.

- **Skew**: Increasing the degree of parallelism increases the variance in service times of parallely executing tasks. Overall execution time determined by **slowest** of parallely executing tasks.

What about "Distributed" Systems?

- Over a wide area network
- Typically not done for *performance reasons*
  - For that, use a parallel system
- Done because of necessity
  - Imagine a large corporation with offices all over the world
  - Or users distributed across the globe
  - Also, for redundancy and for disaster recovery reasons

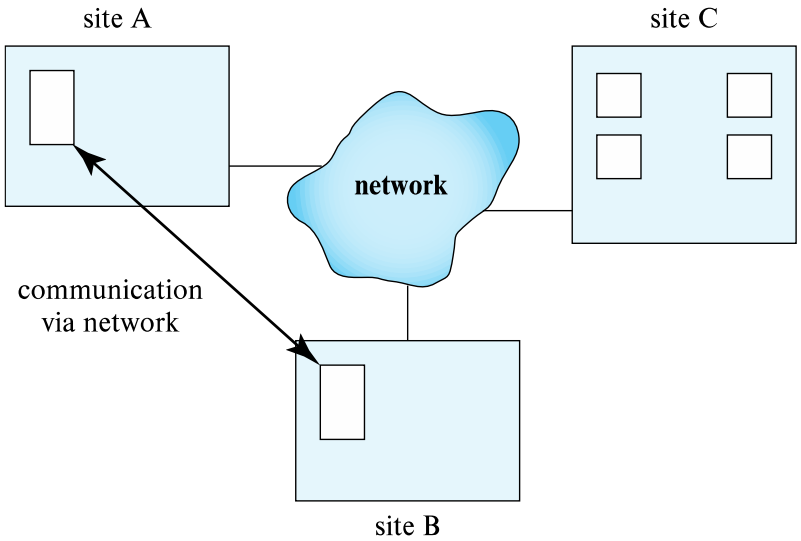
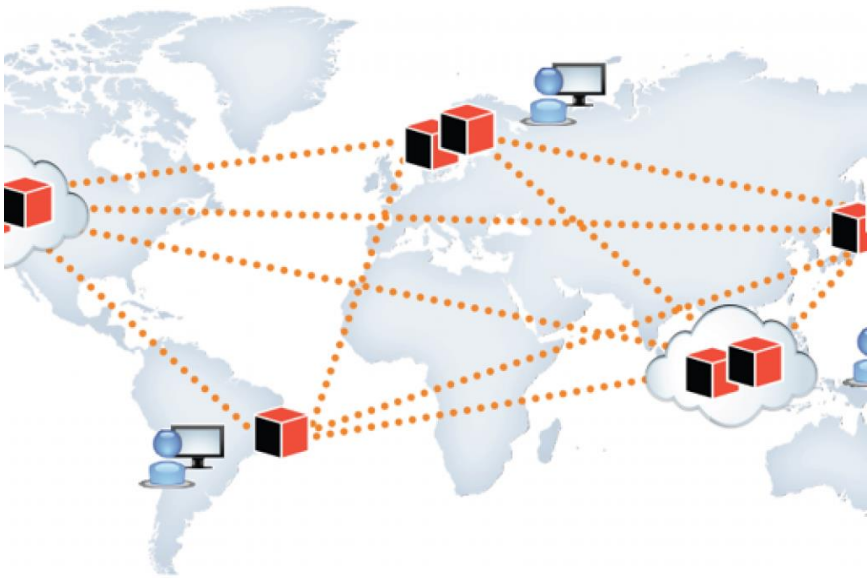


Figure 20.9 A distributed system.



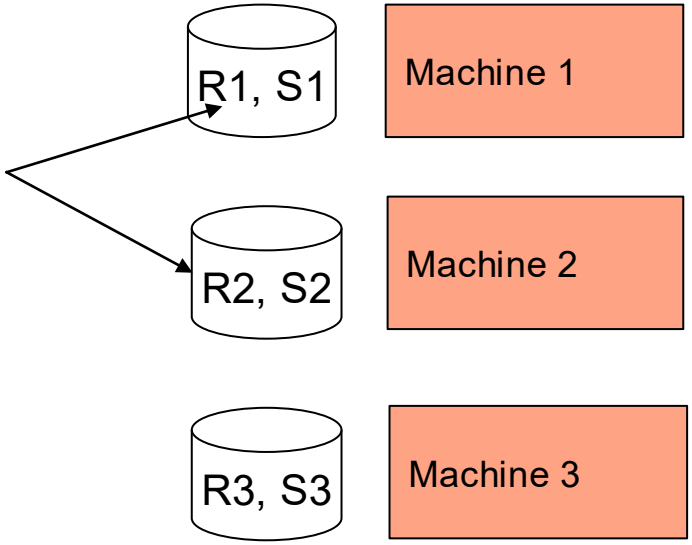
Parallel or Distributed Systems

- Key Questions from Data Management Perspective:
  - How to partition (or “shard”) data across a collection of storage devices/machines
  - How to execute an “operation” across a group of computers
    - In different configurations (shared-memory vs shared-disk vs shared-nothing vs NUMA)
    - Trade-offs and bottlenecks can be vastly different
  - How to execute an “update” across a group of computers
    - Need to ensure consistency
  - How to deal with “failures”

Data Partitioning

- Partition a relation or a dataset across machines
  - Typically through “hashing”
- Advantages:
  - **In-memory computation**: data fits in memory across machines
  - **Parallelism**: simple read/write queries can be distributed across machines
- Disadvantages:
  - **Complex queries:** require combining data across all partitions, especially “joins” are tricky

Partitions of R (Not different relations)

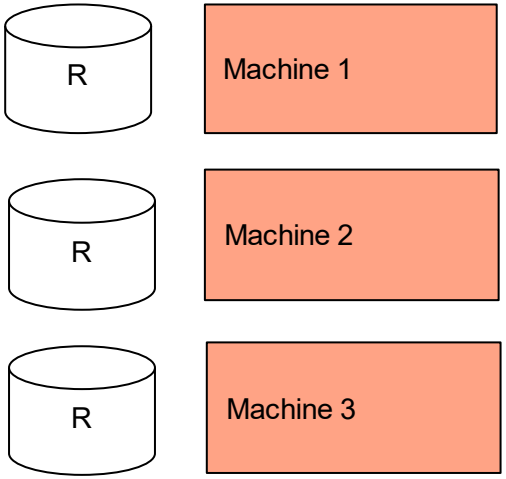


Machine 1 can directly read R1, S1

If it wants R2, Machine 2 must read it and send it to Machine 1

Data Replication

- A data item (file, relation, relation fragment, object, tuple) is **replicated** if it is stored redundantly in two or more sites
- Advantages:
  - **Availability**: failures can be handled through replicas
  - **Parallelism**: queries can be run on any replica
  - **Reduced data transfer**: queries can go to the "closest" replica
- Disadvantages:
  - **Increased cost of updates:** both computation as well as latency
  - **Increased complexity of concurrency control:** need to update all copies of a data item/tuple



Read queries can go to any machine

Write queries must go to "all" machines (if we want consistency)

e.g., what if Application 1 writes to Machine 1, and Application 2 sends its write to Machine 3
-- May result in an inconsistent state

Data Sharding + Replication

- Many data management systems today combine both
  - Partition a dataset/file/relation into smaller pieces and distributed it across machines
  - Replicate each of the pieces multiple times
- This may be done:
  - In a data center with very fast networks, or
  - In a wide-area setting with slower networks and higher latencies
- So need to worry about:
  - Efficient execution of complex queries
  - Consistency for updates
  - Recovery from failures

Failures

- Need to consider:
  - Disk failures: one of the disks (hard drives or SSDs) fails
    - Not uncommon with 10's of thousands of disks
  - Network failures: machines may not be able to talk to each other
  - Machine failure: a machine crashes during the execution of a query or a transaction
- Required guarantees:
  - Shouldn't lose any data if a disk fails
  - Consistency (when making updates) shouldn't be affected if one of the involved machines fails
    - Or if machines are not able to talk to each other
  - Shouldn't have to restart a complex analytics task entirely if one of the involved machines fails

CMSC424: Database Design

Module: Query Processing and Optimization

Parallelizing Operations

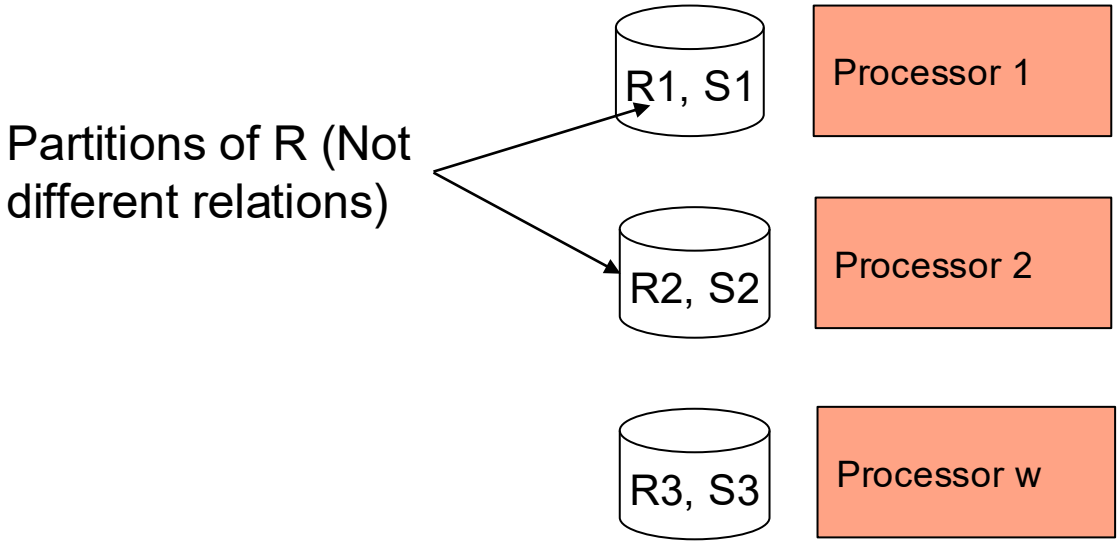
Instructor: Amol Deshpande
amol@umd.edu

Parallelizing Operations

- Book Chapters
  - 18.5, 18.6
- Key topics:
  - Parallelizing a Sort Operation
  - Parallelizing a Join Operation
  - Parallelizing a Group By Operation

Setup

- Assume Shared-Nothing Model
- Relations are already partitioned across a set of machines (will talk about how next video)
- How to execute different operations?



Processor 1 can directly read R1, S1

If it wants R2, Processor 2 must read it and send it to Processor 1

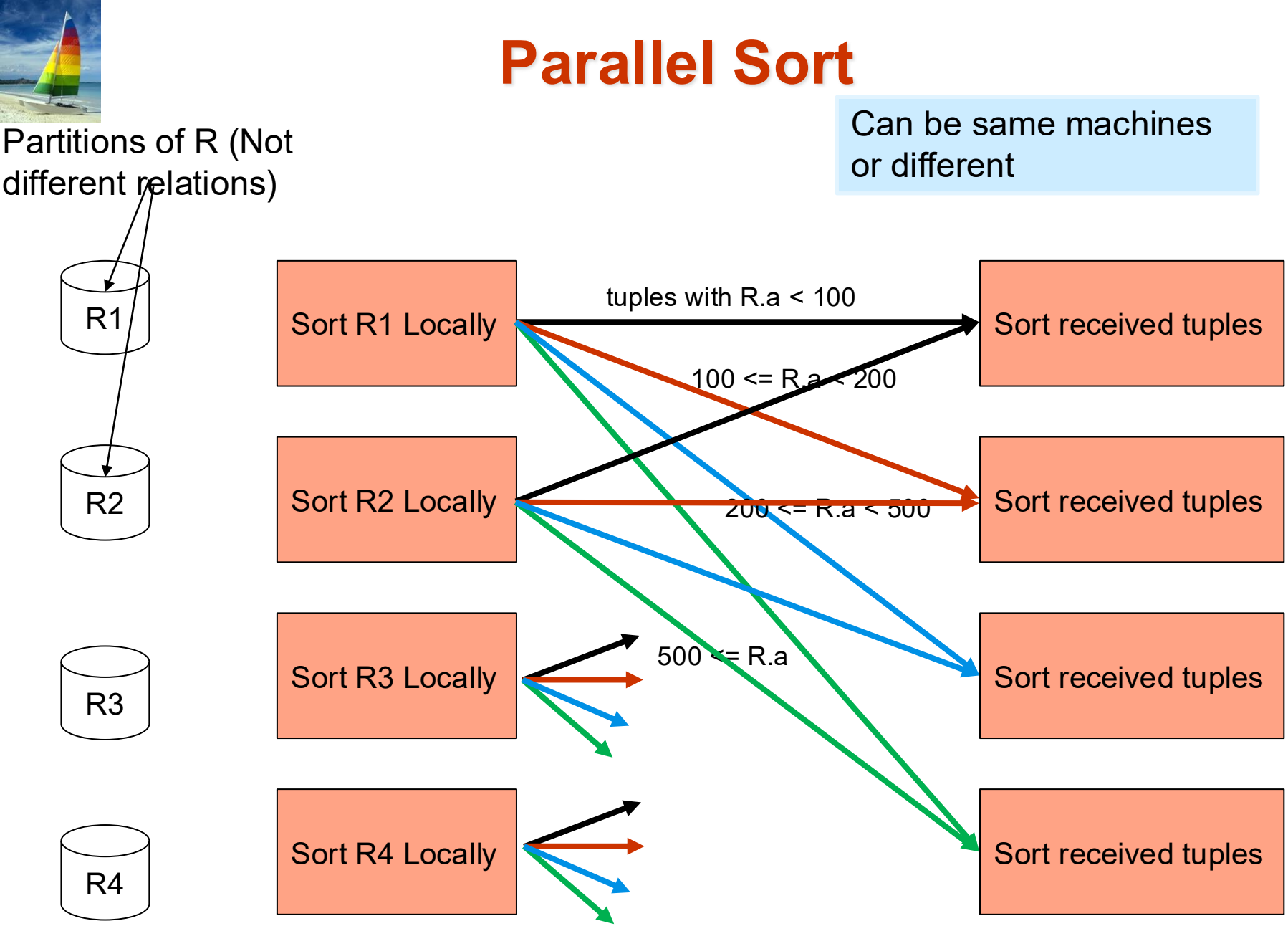
Parallel Sort

1. Each processor sorts a portion of the data (e.g., the data on their local disk)
2. If the data is small enough, all the processors can send it to a single machine to do a “merge”
3. If the data is large, then ”merge” itself done in parallel through range partitioning
    1. Each processor in the merge phase gets assigned a range of the data
    2. All other processors send the appropriate data based on that range partitioning

- In either phase, the processors work by themselves (“data parallelism”) but data must be “shuffled” in between
- Other approaches exist, but basically same steps

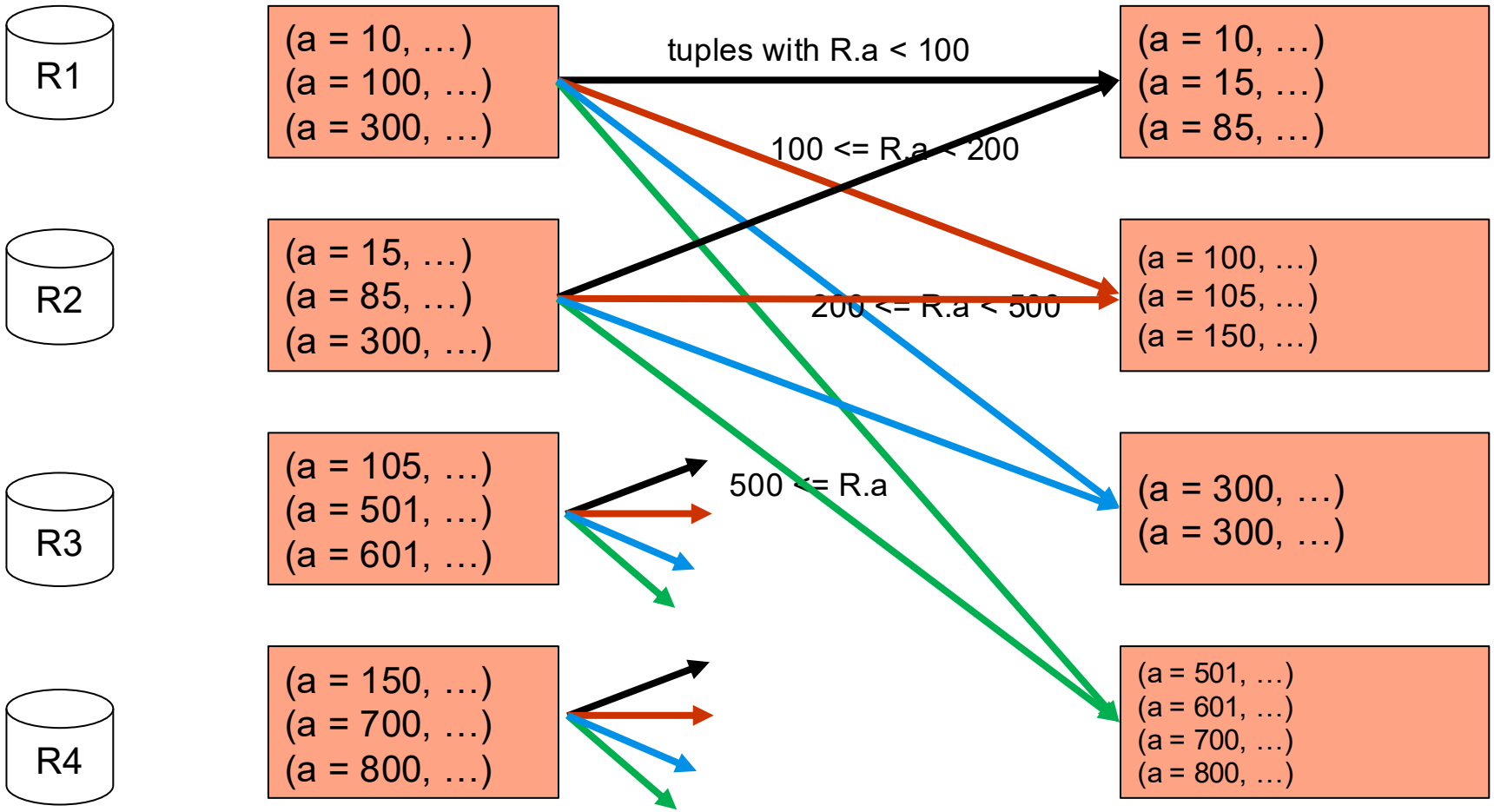
Parallel Sort

Can be same machines or different



Shuffle – typically expensive

Parallel Sort

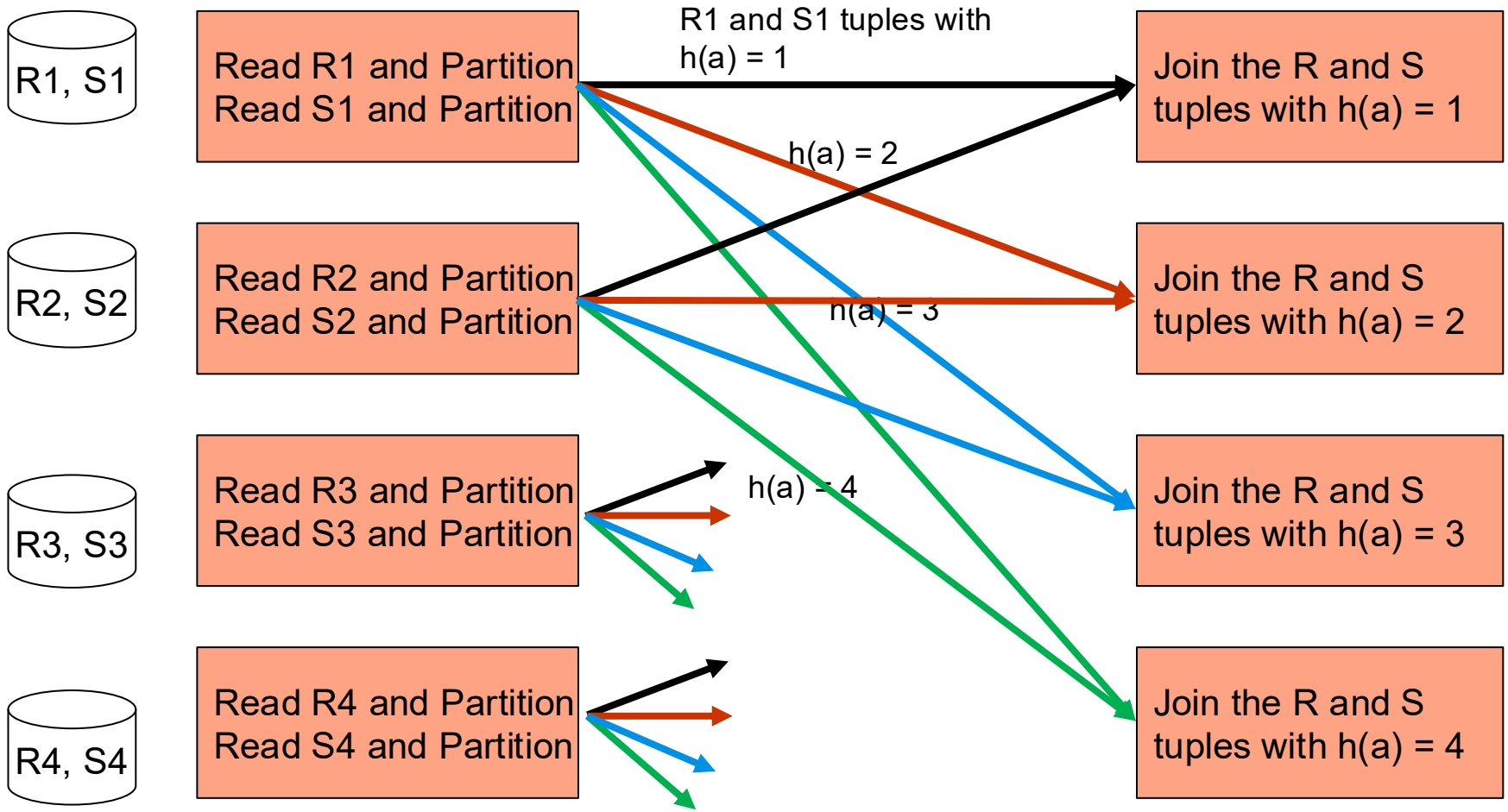


Parallel Join

- Hash-based approach
  - Very similar to how partitioning hash join works (i.e., the variant we saw for the case when the relations don't fit in memory)
  - Most common for equi-joins where hashing can be used
  - Easier to guarantee balanced work

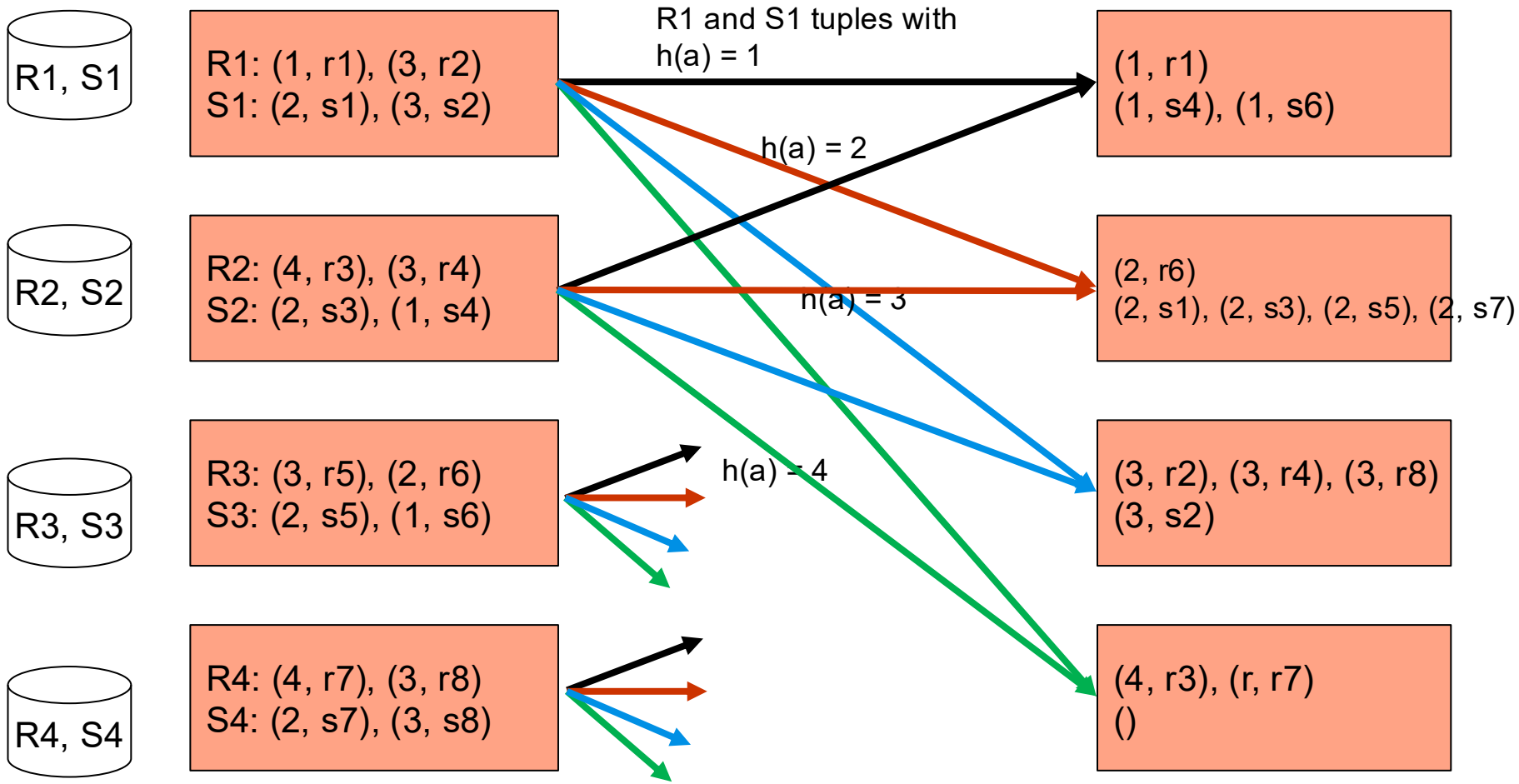
- Sort-based approach
  - Similar to the parallel sort approach
  - Both relations sorted using the same key
  - Same processor used for merging in the second phase for both relations

Hash-based Parallel Join



Shuffle – typically expensive

Hash-based Parallel Join

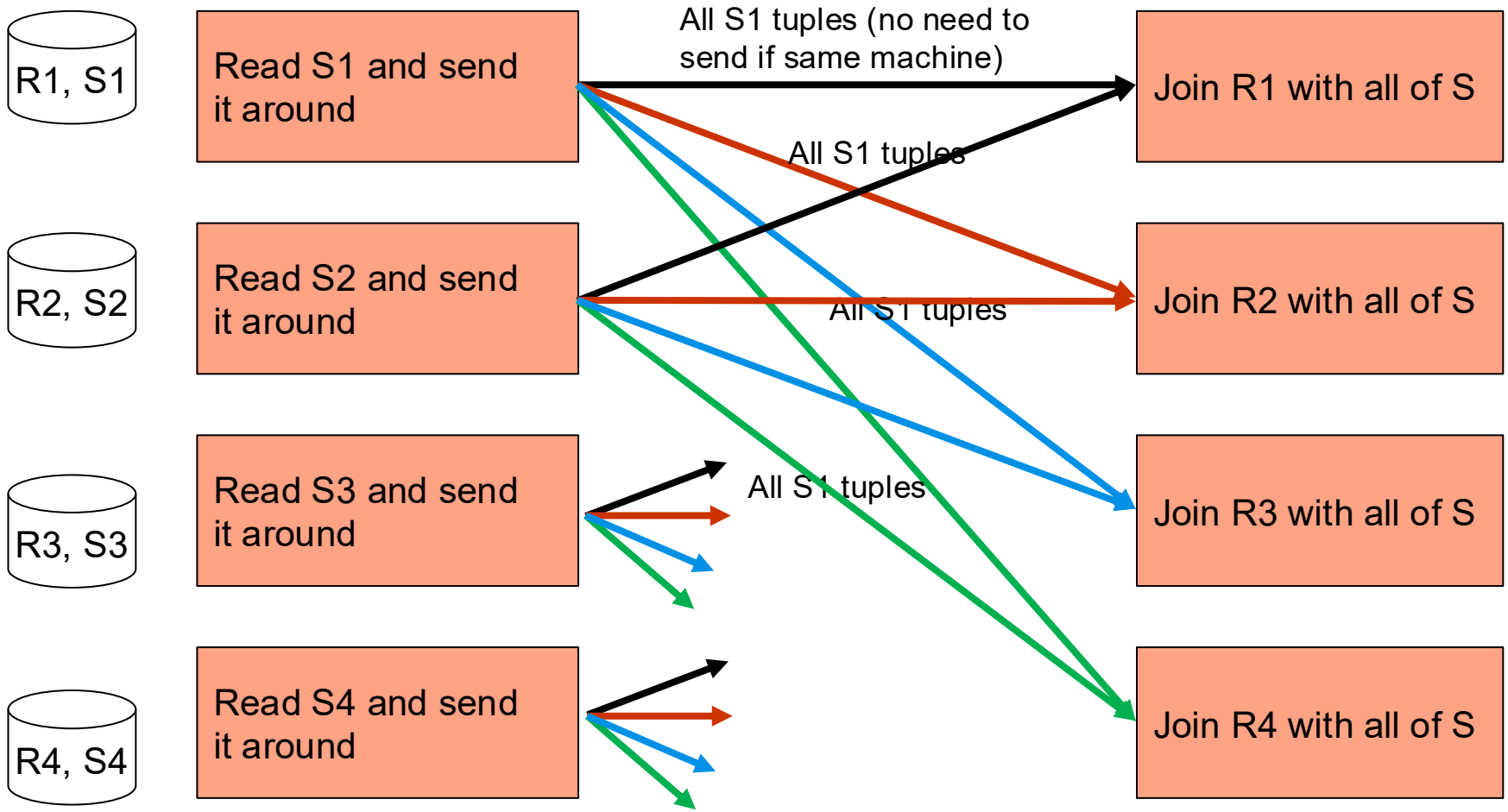


Shuffle – typically expensive

Fragment-and-Replicate Join

- Partitioning not possible for some join conditions
  - E.g., non-equijoin conditions, such as r.A > s.B.
- For joins were partitioning is not applicable, parallelization can be accomplished by **fragment and replicate** technique
- Special case – **asymmetric fragment-and-replicate**:
  - One of the relations, say *r*, is partitioned; any partitioning technique can be used.
  - The other relation, *s*, is replicated across all the processors.
  - Processor P_i then locally computes the join of r_i with all of s using any join technique.

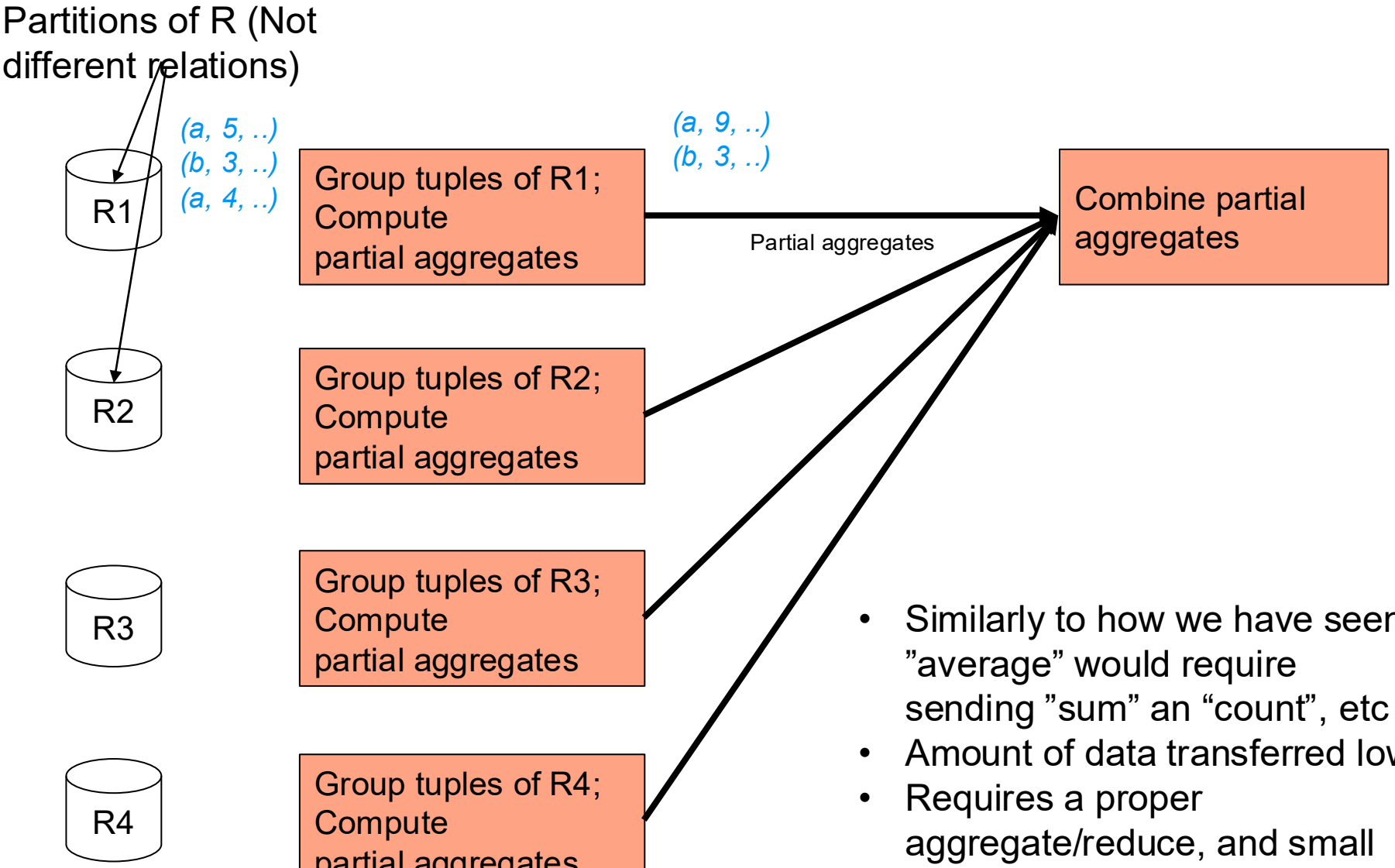
Asymmetric Fragment and Replicate



Grouping/Aggregation

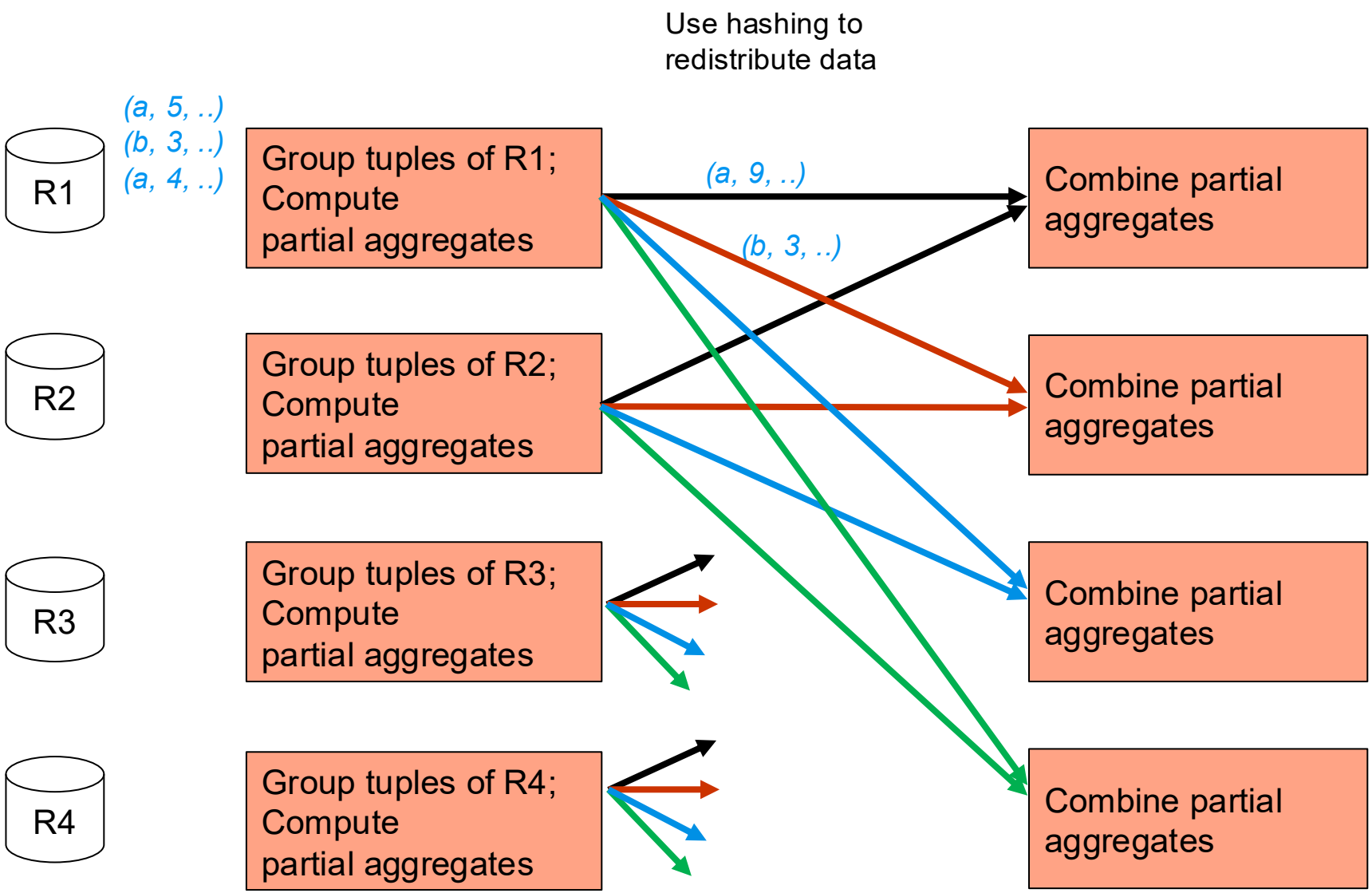
- Very common operation, especially in Map-Reduce applications
  - E.g., grouping by “hostnames” or “words” (as in project 5) or “labels” (in ML context), etc.
  - The idea of distributing data, doing some computations, and collecting results is quite powerful
  - Even ”joins” can be seen as a “group by” operation (you can group the tuples of the two relations on the join attribute, and then compute join)
- Need to differentiate between “groupby” and “aggregate” (”reduce”)
  - **Groupby:** For every value of ”group by attribute” (i.e., “key”), collect all tuples/records with that key on a single machine
  - **Aggregate/Reduce:** Perform some computation on them, typically reducing the size of the data
  - Spark has more granular operations than SQL
- Challenges:
  - Number of keys might be very large
  - Should try to do as much pre-aggregation as possible

Scenario 1: Small # of Groups + Reduce



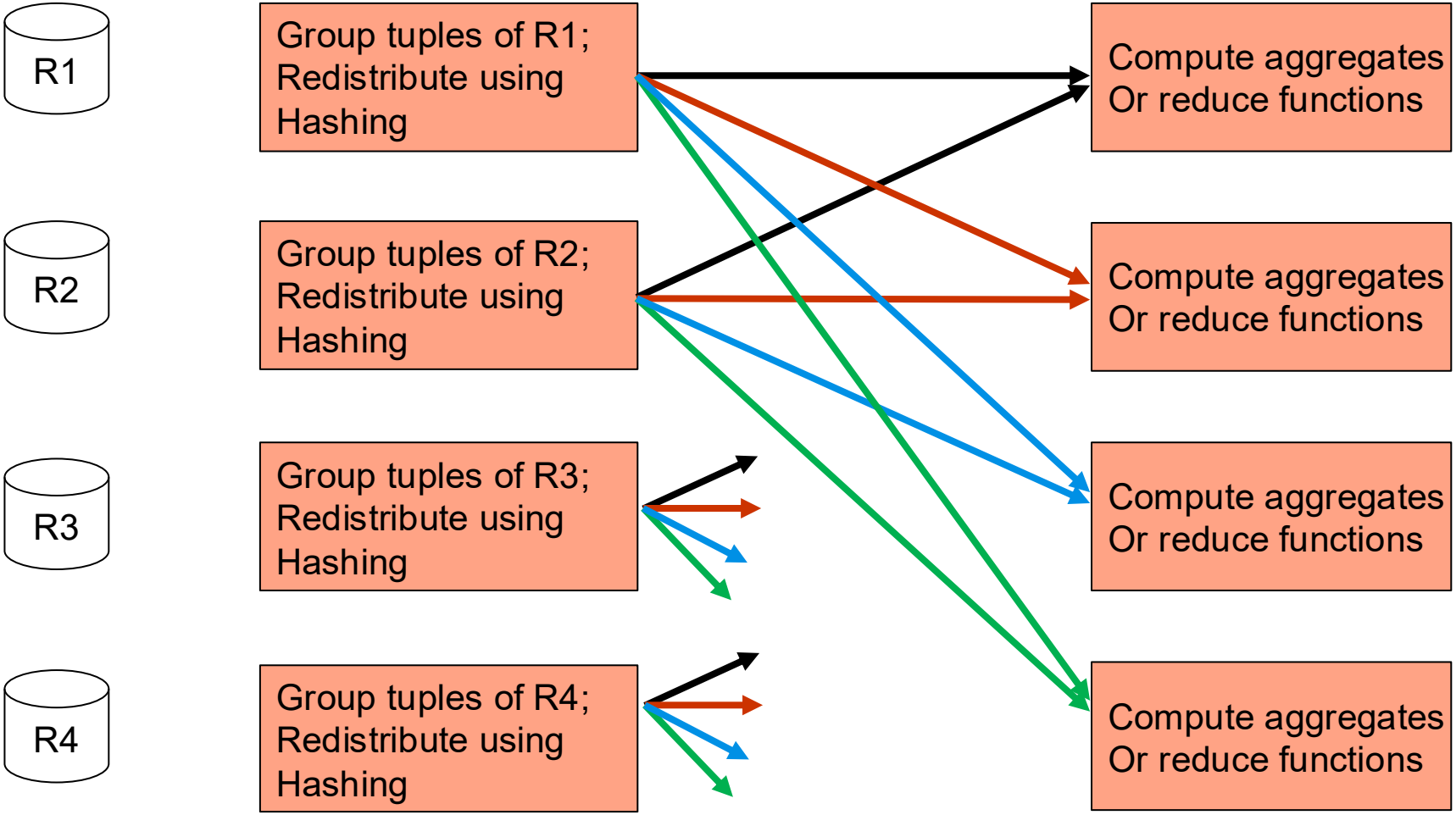
- Similarly to how we have seen, ”average” would require sending ”sum” an “count”, etc
- Amount of data transferred low
- Requires a proper aggregate/reduce, and small number of groups

Scenario 2: Large # of Groups + Reduce



Scenario 3: Large # of Groups + No Reduce

e.g., if we want to compute “median” or some other complex statistics – no “partial aggregation” possible



Other Relational Operations

Selection $\sigma_\theta(r)$

- If θ is of the form $a_i = v$, where a_i is an attribute and v a value.
  - If r is partitioned on a_i the selection is performed at a single processor.
- If θ is of the form $l <= a_i <= u$ (i.e., θ is a range selection) and the relation has been range-partitioned on a_i
  - Selection is performed at each processor whose partition overlaps with the specified range of values.
- In all other cases: the selection is performed in parallel at all the processors.

Other Relational Operations (Cont.)

- **Duplicate elimination**
  - Perform by using either of the parallel sort techniques
    - eliminate duplicates as soon as they are found during sorting.
  - Can also partition the tuples (using either range- or hash-partitioning) and perform duplicate elimination locally at each processor.

- **Projection**
  - Projection without duplicate elimination can be performed as tuples are read in from disk in parallel.
  - If duplicate elimination is required, any of the above duplicate elimination techniques can be used.